FastAPI

Learning Sources
FastAPI Official Docs

Notes-Notes originally created in Jan 2025 & completed July 2026

00. Intro to FastAPI

-Source: https://fastapi.tiangolo.com/

-FastAPI - a modern, high-performance web framework for building Python APIs with type hints

-Features - very high performance, on par with NodeJS and Go, fast to code in, fewer bugs, intuitive, easy to learn, short, robust w/ interactive documentation, and complying with OpenAPI specifications/standards.

-FastAPI requires pydantic (data validation library) and Starlette (an ASGI, async server gateway interface, for building Py async web services)

Install

```
cd <project directory>
python3 -m venv venv
source venv/bin/activate
pip install "fastapi[standard]"
```

Create

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def read_root():
    return {"Hello": "World"}

@app.get("/items/{item_id}")
async def read_item(item_id: int, q: str | None = None):
    return {"item_id": item_id, "q": q}
```

Run

```
$ fastapi dev main.py          # starts dev server

# Serving at: http://127.0.0.1:8000
# API docs: http://127.0.0.1:8000/docs
```

Check it Out
-open *http://127.0.0.1:8000/items/5?q=somequery*

-Response is *{"item_id":5,"q":"somequery"}, where item_id* set as part of call to url (*/5*) and request params set by *?q=somequery*

-Check out headers, also. Notice, server is Uvicorn (an ASGI web server that loads and serves the application).

No permission policies set, yet, also.

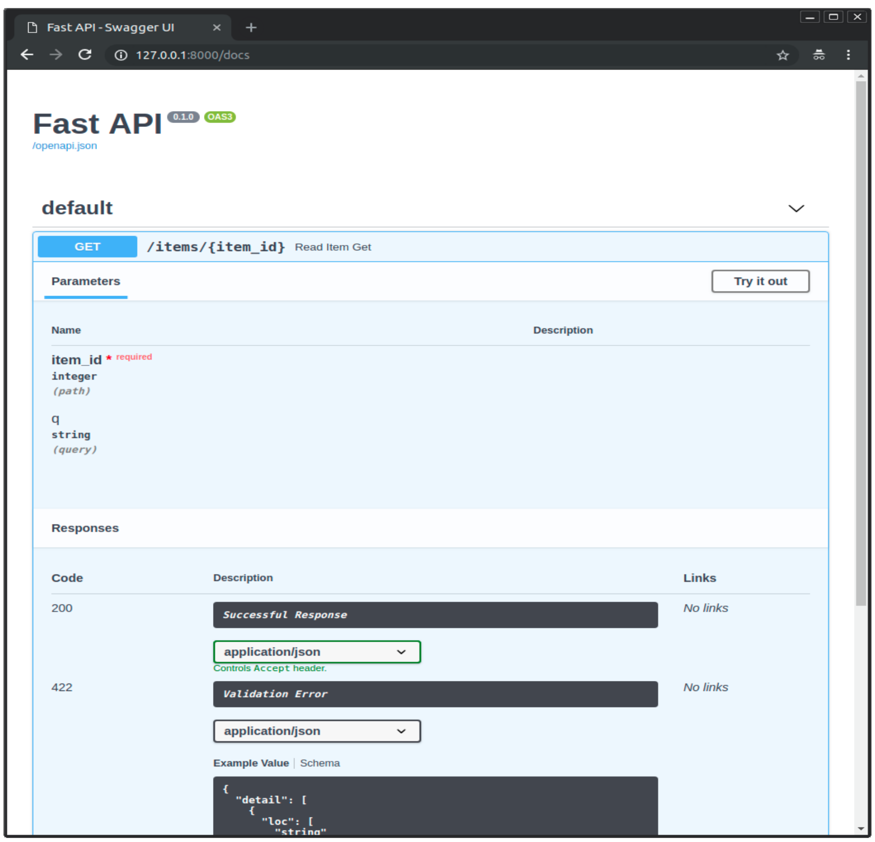
AI Agent Skill
-FastAPI has made a skill available for AI agents, including Claude Code, Codex, etc. to use. Install via:
pip install library-skills

What Just Happened
1. An app was created via creating an instance of FastAPI
2. Multiple HTTP GET paths were added, including one that allowed query params and resource IDs
3. FastAPI was called via querying path on the browser
4. FastAPI returned a JSON response

*Note: *path*, *endpoint*, and *route* all mean the same thing in API context and can be used interchangeably

Interactive Docs
-Go to: http://127.0.0.1:8000/docs

-FastAPI creates interactive OpenAPI UI docs for all paths. The docs show paths and allows you to make sample requests to them, with customized resource ids and params. Doc then shows request URL, response body, response headers, response code, etc.



-Alt UI docs also available via ReDoc at http://127.0.0.1:8000/redoc

Add More Functionality

-In *main.py, can add:* *from pydantic import BaseModel*

```
class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None

@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item):
    return {"item_name": item.name, "item_id": item_id}
```

-Dev server automatically updates, as soon as save code file, so can see new *PUT /items/{items_id}* path and space to enter in detailed request body to call path with:

```
{
  "name": "string",
  "price": 0,
  "is_offer": true
}
```

-Which, when called with valid test data (*{"name": "test_name", "price": 1234, ...}*) , responds with headers, 200 status code, and *{ "item_name": "est_name", "item_id": 1234 }* response body

Summary

-FastAPI lets you declare path request parameters / body as typed function parameters to an HTTP operation path declarations that wraps a standard python function.

-Simple request parameters can be defined using existing python types (ex. *item_id: int*) while complex params can be defined by python data classes (see *Item* example above)

-With each defined path, FastAPI provides:

- data validation - automatic and clear errors when data invalid. Validation even for deeply nested JSON objects.
- editor support - including auto-completion and type checks
- conversion of input data to python data/types, reading from JSON, path params, query params, cookies, headers, forms, and files
- conversion of output data from py data and types, to network data (as JSON), including conversion of py types, *datetime* objects, *UUID* objects, database models, and other types
- Automatically generated interactive docs with two user interfaces (OpenAPI UI and ReDoc)

00. Python Types

Type Hints & Benefits

-Python allows optional "type hints" to declare a type of a variable. Declaring types not only prevents bugs but creates clearer, more fixed models, and gives you IDE hints for code, after var is typed.

-Example of IDE benefits of type hinting: Function takes in untyped param *first_name*. You want to make sure it is upper case before printing it. Is the py function called *upper()* or perhaps *capitalize()?* You enter a '.' after the *first_name* variable, to see if the IDE will list you functions, but it doesn't. If you make *first_name* typed, with *first_name: str*, though, now the IDE sees the var's String methods. Boom, you find the method is called *capitalize()* when you hit '.' after the var, this time.

-Type hinting in function param syntax ex.: *def my_function(first_name: str, age: int)*

-More important than IDE auto-complete benefits is improved data validation, for wrongParamType, etc. otherwise potentially overlooked mistakes.

Py Types

-Need to know Py types: *str, int, float, list, tuple, range, dict, defaultdict, set, frozenset, bool, None*

-If need be, can cast from one type to another. Ex. *str(age)* where *age* is *int*

-Container types (*set, list, tuple, dict*) can also declare the types of items they hold, and can be nested

-Ex. of container w/ typed *items: list[str]*

-"Type parameters" are passed into container square brackets to determine contents' type.

-With tuple, since fixed length, can declare multiple types, each element its own type. Ex. *mixed_tuple: tuple[int, int, str]*. Each element must pass its own type check upon initialization.

-dict takes in two types, for key and value. *dict[keyType, valueType]*. Ex. *dict[str, int]*

Union
-Allows a variable to be one of several types

-Syntax: *my_func(var_name: typeA | typeB):***Default Values & Possibly None**
-Can give function args default values, such as possibly "no value":-Syntax: *my_func(legal_name: str | None = None)*

Classes as Types
-Type can also be a class, in which case args must be instances of that class

Pydantic Models
-Pydantic is a Py data validation library. It allows you to declare complex "shapes" of data classes. All class attributes are typed vars.

-When you create an instance of the class with some values, Pydantic verifies the values match the required attribute types, and converts them to the appropriate type, if possible, then gives you an object with all that data.

-In order to make your class a Pydantic model, it must inherit from the Pydantic *BaseModel* class. Ex. *class MyClass(BaseModel)*

-Ex.

```
from datetime import datetime
from pydantic import BaseModel

class User(BaseModel):
    id: int        name: str = "John Doe"
    signup_ts: datetime | None = None
    friends: list[int] = []
```

```
external_data = {
    "id": "123",
    "signup_ts": "2017-06-01 12:22",
    "friends": [1, "2", b"3"],
}

user = User(**external_data)
print(user)
# User id=123 name='John Doe' signup_ts=datetime.datetime(2017, 6, 1, 12, 22) friends=[1, 2, 3]
print(user.id)    # 123
```

Type Hints with Metadata Annotations
-Py 3.9+ includes *Annotated*, which allows you to include metadata for typehints, where this can provide additional info on a parameter

-Syntax:

```
from typing import Annotated

def say_hello(name: Annotated[str, "this is just metadata"]) -> str:            # type of name is str, plus str has metadata as provided by Annotated
```

Type Hints in FastAPI
-Due to FastAPI's use of type hinting and Pydantic, with FastAPI, you get editor auto-complete, type checks, and data conversion, data validation, type documentation in OpenAPI/ReDoc Docs, and FastAPI can define requirements from query params, bodies, etc.

00. Concurrency and async / await

Overview
-Py version of async code via *co-routines* is enabled by *async* and *await* usage.

Asynchronous Code
-Code that is safe to execute at the same time, even if one *async* part of code requires data from another *async* part of code. Python has a single execution thread. Thus need to use *async* and *await* to enable concurrency.

-In Python, even if one *async* piece of code needs data from the other *async* code, that is okay, due to *await* (concurrency).

-Async example: Computer runs long *async* script that needs external data mid way through. Script starts, and another *async* script starts generating external data. At same time, main script keeps running. When main scripts hits point where it needs external data, if external data still not ready, it will *await* for it, then continue running when it becomes available.

-Common reasons for *async*:

- data from client is being sent through network
- data from application is being sent to client through network
- contents of a disk file being parsed
- file being written to disk
- remote API operation
- database operations

- etc. latency causers

-Synchronous code must finish step to step, in an always repeating single order, where async allows concurrent code to execute.

-Parallelism, like concurrency, also says, "different things, happening, more or less, at the same time," but in a different manner.

Concurrent Burgers
-An example of concurrency:

You go to burger store and order two burgers from a single cashier. The cashier yells back to the cook, "two burgers!," and the cook takes note of the order, while continuing to work on other existing orders. While the cook keeps cooking, you pay, then the cashier gives you change. The burgers are not done yet, so you go sit at a table and chat with your friend. You listen for the cook to announce your order is ready, then go pick it up when it is. You then eat the completed burgers.

Parallel Burgers
-An example of parallelism:

You go to a burger store and order two burgers from a long line of many cashiers. Each cashier is also a cook, and after taking an order and payment, goes back to cook the order.

The burgers are not done yet, but you also have to wait at the counter for them to finish, as your cook does not have order numbers, as he only ever has one order at a time. The cook finishes cooking your food, brings it to you, then you go eat. You do not have time to chat with your friend during cooking, due to waiting at the counter at this time.

Parallelism vs Concurrency
-Parallelism occurs when multiple threads are available, all running across multiple processors (or distinct threads in a multi-core processor).

-Concurrency occurs when tasks can run together, as long as they stop and wait for data (or whatever) when they need it (or need it done) but some other task has not yet produced it.

-Benefit of concurrency over parallelism is that even without multiple threads/processors available, you can still execute other related tasks while waiting. If a lot of waiting, concurrency thus makes sense, so can execute other tasks, while waiting.

-As web applications involve a lot of waiting, concurrency makes sense for many web applications.

-Server waiting on user to send requests. Users waiting on responses to come back. Request-response, is full of waiting. Even JS on the front-end is event-loop driven.

-On flip side, if no waiting required, just a lot of work to be done on different, unrelated tasks, then parallelism makes sense. Ex. eight rooms in a house each being cleaned by their own house cleaner, each working working together, <u>in parallel</u>.

-Parallel problems are, "CPU bound," as the more CPU threads you can run, the more parallel tasks can execute.

-Common reasons for parallelism:

- audio and image processing

- machine learning
- deep learning

FastAPI Concurrency + Parallelism
-NodeJS is also async, as is Go. Starlette (part of FastAPI basis) offers both parallelism and concurrency, thus making FastAPI faster than most NodeJS frameworks, and on par with Go.

-Concurrency comes from FastAPI use of *async* and *await*, which works via an event loop, just like in JS

-FastAPI also allows parallelism, via allowing multiple *Worker* processes to start on application launch, which then allows replication of processes to take advantage of multiple cores, to be able to handle more requests.

async* and *await
-Syntax: *async def my_function(arg: type):* ...
await my_function("my_arg")

-Nice to work with, as this allows the code to still read like it is running sequentially.

-In FastAPI, *async* allows execution on one request to *await*, while FastAPI takes care of another request

-All *async* functions <u>must</u> be called with *await*

-Ex. Async FastAPI Route:

```
@app.get('/burgers')
async def read_burgers():
    burgers = await get_burgers(2)
    return burgers
```

-As any function which contains *await* logic is waiting on an *async* function itself, that function must also be *async*.

-Ex.

```
async def get_data():
    ...


async def get_double_data():
    return [await get_data(), await get_data()]
```

FastAPI Async Basis
-Starlette, which is basis for FastAPI, is based on AnyIO, which makes FastAPI compatible with both Python's standard *asyncio* library and Trio (a Py friendly library for async concurrency and I/O)

-You can directly use AnyIO for advanced concurrency use cases, both within FastAPI apps, and simply as its own library, in non-FastAPI apps

-There's also the Asyncer library, which improves type annotations, gets better auto-completions, provides better inline error handling, etc., compared to using AnyIO alone. Useful for if need to combine async with sync code.

-All FastAPI standard *def sync* functions are run in the single standard Py external threadpool, with returns without awaiting

-A component can own both async and sync functions

Coroutines
-The return of an async function. Similar to a JS promise. Something like a function, that Py can start at some point, and will end at some point, but that might pause, too, whenever an *await* occurs.

00. Environment Variables

-AKA Env var. A variable that lives outside of the back-end language (ex. Py), in the OS, and can be read by the back-ends

-Useful for holding application settings and config

-Can create env var by calling *export MY_NAME="Wade Wilson"*, after which can *echo "Hello $MY_NAME"*. Can also create an *.env* file, to store env var to use within application.

-Get OS env var with:

import os
os.getenv("NAME_NAME", "default_value") *#default for if MY_NAME var not set*

-As env vars are environment-related and stored outside the program, you do not need to commit them to git

-Can create env var for program that only exists during program execution with: *MY_NAME="Wade Wilson" python main.py*

Types and Validation
-Env vars can only be created as strings, and are thus imported into the Py app as strings, too.***PATH* Env Var**
-PATH env var - a special env var used by the OS to find programs to run, such as commands which one might execute via the terminal

-On Linux and macOS, this is */usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin*, where each *:* is a delim for a diff directory to read from

-Should add *Python* to your *PATH* variable, so you can run it as a command, to start a program, etc.

00. Virtual Environments

-Virtual environments create virtual execution environments for programs, where each stores and manages their own dependencies. Any directory can be a virtual environment and all files within it are part of that venv.

-Create a virtual environment:

1. Make a directory and change to it
2. Run *python -m venv .venv* # runs py module *venv* and creates a venv in subdir *.venv*
   # can name *.venv* dir anything, such as *.venvvvvvv*

-Activate virtual environment: *source .venv/bin/activate*

-Check v env is active: *which python*

-Deactivate v env: *deactivate*

-After creating a new *venv*, and activating it, you should upgrade *pip* via: *python -m pip install --upgrade pip*

-Make sure to add *.venv* to *.gitignore*, so it does not get committed

-Once in *venv*, can install requirements directly, via *pip install "fastapi[standard]"* or have a *requirements.txt* file that holds requirements to install via *pip install -r requirements.txt* :

```
// requirements.txt
fastapi[standard]==0.115.6
pydantic==2.10.4
```

-Once in v env, any commands run inside are run in v env, so if you run *fastapi dev main.py* in venv, will execute a *fastapi* instance in that venv.

-Make sure to configure your IDE to use your project's venv, if it does not automatically, to ensure you get auto-completion from dependencies, ability to step into dependency code, etc..

-If you do not use a venv, *pip* installs are installed to the global *Python environment*, which wastes OS space and does not properly encapsulate dependencies on a project-by-project basis. What happens, also, if two programs require the same dependency, but different versions? This also makes it difficult to manage dependencies, when all installed in global env.

-When you activate a *venv*, as part of setting that venv active, *./venv/bin* is added as part of v env *PATH*, putting the *venv* directory at the start of *PATH*, so it is the first directory commands try to execute from. Ex.

/home/user/code/awesome-project/.venv/bin:/usr/bin:/bin:/usr/sbin:/sbin

-Plenty of other ways to manage projects also exist, like the uv tool, which is a tool to manage the entire project, package dependencies, virtual envs, etc.. or Docker Compose.

01. FastAPI First Steps

Check it Out

-Start by creating a sample FastAPI project directory, then creating a Python venv within it, then activating venv, and installing FastAPI from within venv via: *pip install "fastapi[standard]"*

-The most basic FastAPI application contains an instance of the app and a single route within it:

```
#main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

-Once you have a project with some basic FastAPI code, you can start the FastAPI dev server via command: *fastapi dev main.py* , which spins up a live server and gives you a URL to the server, and the auto-created OpenAPI/ReDoc docs for the API.

-Local server at: http://127.0.0.1:8000
-Docs at: http://127.0.0.1:8000/docs

OpenAPI
-Schema - a definition of the layout for something
-API Schema - the paths of the API, the possible params an API path will take, etc.. In FastAPI, the schema is based on OpenAPI specifications.-Data Schema - the shape of the data, such as JSON content. FastAPI uses JSON schema, where attributes are also typed. API schema uses JSON schema as part of definition.

-API Schema is stored in *openapi.json* file created and updated by FastAPI, which can be viewed locally at: http://127.0.0.1:8000/openapi.json

-The OpenAPI schema powers the interactive docs. Many alts for interactive docs exist, and all can be integrated easily into a FastAPI app by using the OpenAPI schema as input.

-The OpenAPI schema is also useful for auto-generation of code, for clients that communicate with FastAPI, such as a front-end.

Starter Details
-Based on the code seen in "Check it Out" section above

- *@app.get("/")* creates a path decorator for the root path. A decorator in FastAPI is the pattern in which you place behavior inside a wrapper for various HTTP operations, to give logic to those operations, for a specific path. FastAPI calls these path operation decorators.

-A path is a specific route in your API. For example, in *https://example.com/items/foo*, the path is */items/foo*. A path can also be called a "route" or "endpoint." Paths are the main way to separate concerns and resources in a REST API.

-Each path in FastAPI is based on a HTTP operation, including *POST, GET, PUT, DELETE, PATCH, HEAD, OPTIONS,* and *TRACE*. Each path can take in one or more of these operations, each with their own behavior.

-Ex. of path operation decorators: *@app.post(...), @app.put(...), @app.delete(...)*

-There is no enforcement for how you use each HTTP operation, but they should generally follow expectations for use. For *GraphQL* APIs, you'd generally perform all actions via *POST*.

-Path operation decorators sit directly above the function def, as seen in previous example.

-Functions for path operation decorators can be *async* function, or just regular synchronous, depending on need.

-API paths can return anything, from *list* to *dict*, to *str*, to *int*, to more complex Pydantic models, to ORMs. For example, if you return *1*, raw data return will simply be *1*.

Basic FastAPI Recap
1. Import FastAPI
2. Create an *app* instance

3. Write a path operator decorator using a decorator like *@app.get("/mypath")*
4. Define a path operation function underneath the function (sync or *async)*
5. Run the dev server with *fastapi dev entry.py*

02. Path Params

Path Params
-Path params are values passed into the path when it is called. The path param then is also available inside the wrapped path function. Ex.

```
@app.get("/items/{item_id}")
async def read_item(item_id):
    return {"item_id": item_id}
```

-Note the *{}* in the above param. This is where the path param goes. Same syntax as typical python f-string, but without the *f*.

-Path params are usually used to <u>identify a resource or set of resources</u>

Typed Path Params
-Path params can also be given a type, via typical typing of the param inside function signature. Ex. *async def read_item(item_id: int):*-Typing params is pretty much always a good idea, due to better error checks, easier test writing, IDE auto-completion, etc.

-Types are also maintained in returns of FastAPI. Hence, in the above example, the return of calling with *item_id*=3 is *{ "item_id": 3 },* where 3 is an *int*, not a *str*.

-If you try and pass in the wrong type, FastAPI will auto-generate an error object for the return, with a descriptive error message, such as *"msg":"Input should be a valid integer, unable to parse string as an integer"*, and also denoting the error type, path, and input given. Very useful for debugging.

-Interactive FastAPI docs will also show required param types, and display UI errors, if wrong type passed.

-<u>Pydantic</u> provides the data validation

Path Declaration Order
-When you define multiple of the same path operation decorator HTTP types in the same file, order matters, as, if a call could match multiple paths, whichever path is defined first is hit.

-Example:

```
@app.get("/users/me")
async def read_user_me():
  return {"user_id": "the current user"}


@app.get("/users/{user_id}")
async def read_user(user_id: str):
  return {"user_id": user_id}
```

if call above with *mydomain.com/users/me*, FastAPI could see this as to call the */me* path, but could also see *me* as a *user_id*. Hence, whichever path was defined first would be

called.

-As such, you should also not redefine paths using the same path location (ex. *@app.get("/users")* and *@app.get("/users")*), as the first one would always be hit, and the second one just never used pointless code.

Predefined Path Param Values
-If a path param should only accept certain values, you can define these using an *Enum*, via class which inherits from Py *Enum*.

-Ex.

```
class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
async def get_model(model_name: ModelName):
    if model_name is ModelName.alexnet:
        return {"model_name": model_name, "message": "Deep Learning FTW!"}

    if model_name.value == "lenet":
        return {"model_name": model_name, "message": "LeCNN all the images"}

    return {"model_name": model_name, "message": "Have some residuals"}        # example return
{  "model_name": "alexnet",  "message": "Deep Learning FTW!" }
}
```

-Note in above example, that path param has same type as created model value. As such, if pass in an invalid Enum name, where name is not included in Enum (ex. *"rogernet"*), FastAPI will return a type error message for it's response.

-When give a param a predefined set of values, API Docs will also generate a *select* menu for this param, only allowing these types in interactive UI.

-Note, in above example, that you can get the value of the Enum name passed in by calling *.value* on it.

-Can also return Enum names (*model_name* in the above example), which are then type cast to string.

Paths as Path Params
-What if you want to make a file path itself a path param, such as */files/{file_path}* taking in *home/johndoe/myfile.txt* ? Rejected! This creates hard to test and define scenarios and OpenAPI does not allow this.-If you do wish to do a path value as a path param, use an internal tool from Starlette to add support for this.

Path Params Recap
-Thanks to FastAPI path param typing, you get:

- Editor support for error checks, auto-completion, etc.
- Data "parsing"
- Data validation
- API annotation and auto docs

03. Query Params

Query Params
-Query params allow you to make more detailed requests than path params alone. Any function params not declared as part of the path (i.e., not in {} in the path string) defined within a wrapped path function sig are query params.

-Query params are <u>usually used to filter or sort a resource or set of resources</u>

-Example:

```
fake_items_db = [{"item_name": "Foo"}, {"item_name": "Bar"}, {"item_name": "Baz"}]

@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]

# example call mydomain.com/items/?skip=0&limit=10 # returns fake_items_db[0:10], i.e. all 3 items here
```

-Query params in a HTTP address are defined after a *?* char, which follows the path, and which each param is separated by an *&* char, with values assigned to names via = char.

-Ex. *mypath/?skip=0&limit=10* *#skip, with value 0, is first query param. limit, with val 10, is second.*

-All query params, as part of the URL, are strings, but when they are declared types within the path function sig, they are auto cast (and validated) into the proper type.

-The same editor support, data "parsing", data validation, and auto docs features available to path params via FastAPI also apply to query params

Default & Optional
-As query params are not a fixed part of the path, they are often optional, and thus can have default values for if no value provided. Typical Py syntax of *my_var: type = default_val*, for these.

-Params can also be optional, in which no default value is provided, and if value not passed in for param, value is simply *None*

-Optional param syntax varies depending on Py version:

- Py < 3.10: *from typing import Optional* *(my_var: Optional[type] = None)*
- Py >= 3.10:
    - *(my_var: type | None = None)*

Type Conversion-*bool* query params can be converted from numerous values. 1/0, True/False, true/false, on/off, and yes/no will all type cast to a proper True False *bool* value.**Required Params**
-Any param without a default value will be required. Not including it in the request will result in an error object being returned.

-Required and default params can be mixed, as per standard Py.

Enum
-In the same way Enum can be used to define "allowed value" params for path params, so can they be used for

query params, also.

04. Request Body

Request Bodies
-Request query params can be used for simple data retrieving or filtering (ex. *http://example.com/api/data?id=123*). All data sent via query params is also visible through the URL itself.

-Request body data is not visible via URL and is often used for sending larger or more complex amount of info, often in JSON, XML, etc. form.

-Ex. may use path param to get specific object, query param to filter a list of objects, and request body to POST an object

-Request bodies <u>also key for POST, PUT, PATCH, etc.</u>

-Data models in FastAPI inherit from Pydantic *BaseModel* class
-Example:

```
from pydantic import BaseModel
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

-After create Pydantic *BaseModel* inheriting class, can pass it in as a data model to FastAPI endpoint function, using model class name as type.

Request Body Docs
-FastAPI performs the same type conversion, data validation, type checking, JSON OpenAPI schema generation, auto doc generation, etc. on request bodies as it does path and query params.

Editor Support
-Also get type hints, error checking, and auto-completion in editor from declaring data classes. Thanks again to Pydantic for this.

-If using PyCharm for IDE, check out Pydantic PyCharm Plugin for even better Pydantic model support.

Using the Models
-Once a request body model has been passed into endpoint function, can access all props in it as you normally would (ex. *product.price*), then use to update DB data, etc.

Mixing Params
-Can take in path params, query params, and request body all at same time. To differentiate which is which:

- If endpoint function param also declared in path (ex. "/items/{<u>item_id}</u>"}, then function param is path
- If endpoint function param is primitive, then query param
- If endpoint function param is Pydantic model, then is request body

-Ex.

```
# item_id == path param, item == request body, q = query param

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item, q: str | None = None):
    result = {"item_id": item_id, **item.dict()}

    if q:
        result.update({"q": q})

    return result
```

-Can also use *Body* parameters, instead of Pydantic models for request bodies. More on this later

Putting it all Together
-*/this/is/my/path/{entity_id}/?query=param&another=12*

-Path - */this/is/my/path/* - the API endpoint you are hitting-Path Param - *{entity_id}* - typically a unique identifier(s)
-Query Params - *?query=param&another=12* - typically used to sort and filter entities-Request Body - Not seen in URL. Part of HTTP request itself. Typically larger data for C & U.

-User makes a request to a specific path. The path could be general, such as just getting all items, or it could be for a limited number of item(s). When getting specific item(s), path param is used to get specific entity(ies). Query params can this pass in additional vars to help sort, limit, or filter results. Sensitive data and large data, like in POST and PUT/PATCH requests, is sent via the request body.

-FastAPI paths are defined by function wrappers which also define actions for the path based on HTTP verbs. Path params are defined as part of the wrapper path. The wrapped function knows which of its signature params is the path param as that param also defines the path param identifier in the path (ex. *{entity_id}*). If an param is a primitive, it reads it in as a query param. If a param is an Pydantic model, it sees this param as a request body.

-Example: *@app.patch("/items/{item_id}")*
async def set_offer_if_item_expensive(item_id: int, item: Item, expensive_price: float) -> *Item:*
 # item_id arg == {item_id} part of path --> *path param*
 # item is type Item == Pydantic model --> *request body # expensive_price == primitive* --> *query param*

05. Query Params & String Validators

***Query* Validation**-Able to perform additional validation on query params easily in FastAPI. Done via a combo of *Annotated Python module* and FastAPI *Query*: *from typing import Annotated*
from fastapi import Query

@app.get("/items/")
async def read_items(q: Annotated[str | None, Query(max_length=50)] = None): # q can be str or None AND must be 50 chars or less ...

-*Annotated* - allows you to include metadata for typehints, where this can provide additional info on a parameter.

It both sets the type of the param and gives additional data for param, like setting validation with Query() above. *q: str | None = None   q: Annotated[str | None] = None# both mean the same thing-Query* - when pass a *Query* to the *Annotated* object it enforces this query on the path function param, hence having thus both type checking and more complex validation.-If request fails query validation, FastAPI will raise a clear error stating invalid client data. Param will also show the query validation in OpenAPI (Swagger) docs.

-Additional Query validations:

- *min_length=[int]*
- *pattern="^fixedquery$"*

Required & Default Values
-Can of course use default val besides *None* too: (*q: Annotated[str | None, Query(...)] = "default")* . And if don't want default value, just don't give it one in sig: (*q: Annotated[str, Query(...)])* .

-Can force client to send a default value, even if *None* by setting *None* as an optional type, but not giving arg default value via *= None* also. This forces client to send value w/ request even if val is *None*.

Query Param Lists
-If you define a query param as a list you can pass in multiple values for the param throughout a request:
@app.get("/items/")   async def read_items(q: Annotated[list[str] | None, Query()] = []):
query_items = {"q": q}
return query_items   # http://localhost:8000/items/?q=foo&q=bar   <-- q: ["foo", "bar"]

-If want specific size for list, good time to use *min_length* and *max_length Query* validations**General Param Metadata**
-*Query* can take in other args as well, such as:

- *title="Query title"*
- *description="Description of query string and some validation or whatever"*
- *alias="another-name-that-query-param-can-be-requested-as"*

-Ex. ... *Query(title="Query Max", description="Ensure max query size not passed", max_length=75)]***Deprecating Params**-Not something you should do very often, but if you do, notify clients well in advance, via *Query(deprecated=True)*, which will then update docs to show param as deprecated

Hiding Params in OpenAPI
-Can also hide that a param exists for API in OpenAPI docs via: *Query(include_in_schema=False)*

Custom Validation
-Pydantic also provides a collection of custom validators, such as *AfterValidator*. This can also be passed into *Annotated*, instead of *Query*, to perform custom call-back function query param validation.

from pydantic import AfterValidator   …   data = { ... }
def some_validation_function(validate_me: type):
.…   @app.get("/items/")   async def read_items(id: Annotated[str | None, AfterValidator(some_validation_function)] = None):   ....

06. Path Parameters and Numeric Validations

Path Validation-While *Query* FastAPI module allows you to perform validation on query params, FastAPI's *Path* module allows path validation.

-Ex.
from fastapi import Path
...

@app.get("/items/{item_id}") async def read_items(item_id: Annotated[int, Path(title="The ID of the item to get")]): ....

-*Path* works just like *Query* and takes in the same possible args (*min_length*, *max_length*, *title*, etc.) as *Query*.

Number Validation
-Can pass in args:

- *gt=1 – greater than 1*
- *ge=0.5 – greater than or equal to 0.5*
- *lt=1.2 – less than 1.2*
- *le=100 – less than or equal to 100*

-ex. *Annotated[int, Path(ge=0, le=100)]*

-Note that the above number validators can be also used to validate query params as args to *Query*

07. Query Param Models

Query Param Models
-Query param models - allow you to define your query params via a model

-ex.
```
class FilterParams(BaseModel):
    limit: int = Field(100, gt=0, le=100)
    offset: int = Field(0, ge=0)
    order_by: Literal["created_at", "updated_at"] = "created_at"
    tags: list[str] = []
```

-When you set the param type as a Pydantic model, then attach a *Query* to it with *Annotated*, FastAPI defines expected query params by model-This should be preferred to defining params inline, as it allows proper separation and DDD layers, plus makes endpoint definitions less verbose and allows re-usability

-*Field* another Pydantic data model base class (like *BaseModel)* that takes in a variety of possible args such as *default*, *alias*, *strict*, *repr*, and constraints like the

-Query params defined by model props show in OpenAPI docs just like inline defined params

-Can also forbid params from being submitted as part of query via param model prop of:
model_config = {“extra”: “forbid”}   # error raised if client submits params besides model props
-Note that cannot use Pydantic models for path params, as path params are primitive and thus do not need models.

08. Body – Multiple Params

Multiple Params
-FastAPI knows a Path type param is a path param, a Query type param is a query param, and a BaseModel param is a request body.

-Query and request body params can both be optional by giving a default value of None, but path param is required.

-Endpoint can take in multiple body params, too: *@app.put("/items/{item_id}")*
async def update_item(item_id: int, item: Item, user: User):
...
-All separate body objects passed into the endpoint are combined into a single FastAPI JSON object, where each request body passed in lives in this:

{
“item”: {   “item_prop”: “value”, ...   },
“user”: {
“user_prop”: “value”, ...   }   }

Body
-Query params can have extra data for them defined with FastAPI’s *Query* module. Path params with *Path* module. Request body params with *Body* module. Ex.

from fastapi import Body

@app.put("/some_route")
async def update_item(request_body: Annotated[int, Body()]   ...

Primitive Value Request Bodies
-If want to pass in a primitive (int, float, str, etc.) instead of an object to endpoint also can do this:

@app.put("/some_path")
async def update_item(item: Item, user: User, importance: Annotated[int, Body()]):   ...

-FastAPI thus sees this as part of the request body, as just another request body param, instead of seeing it as a primitive and thus treating it as a query param.

Embedded Body Param
-Typically when have single request body param passed into endpoint (ex. *Item*) the body param is expected to look like: *{ "item_prop": "value"* .... *}*

-But you can instead have the prop nested inside the body, referenced by its own name, as an object in the outermost body layer, via *Body(embed=True)*:

```
@app.put("some_route")
async def do_something(my_model: Annotated[MyModel, Body(embed=True)]):
```

```
{
  "my_model": {
    "item_prop": "value"        ....
  }
}
```

09. Body – Fields

Field Metadata
-In the same way can declare extra data on query, path, and request body params via *Query*, *Path*, and *Body* modules, the same can be done on *Field*.

-With field, no need to wrap in *Annotated* though:

```
description: str | None = Field(default=None, title="The description of the item", max_length=300)
```

-Field can take in all the same params as *Query*, *Path*, etc. (*le*, *gt*, *min_length*, *description*, etc.)

10. Body – Nested Models

Collection Props
-Pydantic models can contain props which reference collections of elements, such as *list[str]* or *set[str]*, etc.

-ex.

```
class Item(BaseModel):      tags: set[str] = set()
  weights: dict[str, float]
```

Nested Models
-Models can contain models as props

-ex.

```
class Image(BaseModel):
    url: str
    name: str
```

```
class Item(BaseModel):
    image: Image | None = None
```

-Model within model props get the same editor support, data conversion, validation, documentation, etc. like any other model in FastAPI

-Models can be deeply nested too. A model contains a model which contains a model, etc..

Special Types
-Pydantic provides a variety of special types, many of which inherit from existing types. For example, *HttpUrl* which verifies that the prop is a valid URL and is documented as URL in OpenAPI schema.

-Syntax:
```
from pydantic import HttpUrl                ....
class Image(BaseModel):
    url: HttpUrl
```

11. Declare Request Example Data

Request Example in Model
-Can define example inside model definition by adding *model_config* prop, which contains a *json_schema_extra* object, which contains and *examples* list of example objects:

```
model_config = {
    "json_schema_extra": {
        "examples": [
            {
                "name": "Foo",
                "description": "A very nice Item",
                "price": 35.4,
                "tax": 3.2,
            }
        ]
    }
}
```

-This example(s) will then show as example(s) in API docs

-Defining model schema examples this way is nice as any endpoint which uses the model as it's schema has this example**Param Module Examples**
-*Field* can also provide an example for primitives via *examples* param:

```
description: str | None = Field(default=None, examples=["Large red apple from IN"])
```

-Can also do the same with *Query*, *Path*, *Body*, *Header*, *Cookie*, *Form*, and *File* modules. Ex.
```
@app.put("/items/{item_id}")
    async def update_item(
```

```
    item: Annotated[
        Item,
        Body(
            examples=[
                {
                    "name": "Foo",
                    "description": "A very nice Item",
                    "price": 35.4,
                    "tax": 3.2,
                }
            ],
        ),
    ],
):
```

-Can include multiple examples (inside) *Param* inheriting objects and in definition inside model

OpenAPI Examples
-Can also include examples built for OpenAPI specifically (include more fields inside example like *summary*, *description*, and *value*, which contains actual prop example data:

```
Body(
    openapi_examples={
        "normal": {
            "summary": "A normal example",
            "description": "A **normal** item works correctly.",
            "value": {
                "name": "Foo",
                "description": "A very nice Item",
                "price": 35.4,
                "tax": 3.2,
            },
        },
)
```

-Not much benefit to this for most use cases versus just given standard FastAPI examples

12. Extra Data Types

Additional Data Types
-*UUID* – Universally Unique Identifier common for ids in DB repr as *a* str in requests/responses. Import from *uuid*.

-*datetime.datetime* – py date and time

-*datetime.date* – py date

-*datetime.time* – py time

-*datetime.timedelta* – py timedelta

-*frozenset* – same as set but immutable

-*bytes* – py bytes

-*Decimal* – standard py decimal, handled same as float

13. Cookie Params

-You can tell the endpoint that data being sent into it is a cookie header via:
@app.get("/items/")
async def read_items(ads_id: Annotated[str | None, Cookie()] = None): ...-This says to look for a cookie named *ads_id* being sent over as a cookie header on the incoming request to the endpoint. It will contain a string (or *None*) value.

-Setting a cookie will be covered later in a doc. This just covers how to handle receiving a cookie as part of request.

-Cookies show in docs

14. Header Params

-Define the same way as other *Param* inheriting classes like *Query* and *Cookie*:
@app.get("/items/")
async def read_items(user_agent: Annotated[str | None, Header()] = None):
....

-Headers are commonly named using hyphens (ex. *user-agent*) but Python requires underscores for word separation in var names (ex. *user_agent*). Header thus auto-converts any hyphens in the header name sent to underscores.

-Can disable underscore conversion by passing in param *convert_underscores* as *False* to *Header:*
Annotated[str | None, Header(convert_underscores=False)]

-Can also receive duplicate headers, meaning same *Header* w/ multiple values, just like with *Query* by declaring the *Header* type as a collection:
Annotated[list[str] | None, Header()]

-Headers show in docs

15. Cookie Param Models

Cookie Models
-If reusing a cookie across multiple endpoints, it's useful to declare a cookie model which inherits from *BaseModel* then declare the param as this type when annotating it as a *Cookie* param:

```
class Cookies(BaseModel):
    session_id: str
    facebook_tracker: str | None = None
    google_tracker: str | None = None

@app.get("/items/")
 async def read_items(cookies: Annotated[Cookies, Cookie()]):       ...
```

Forbid Extra Cookies-Not a very common action, but you may wish to restrict the cookies the API receives by including a prop in the cookie model class:
model_config = {"extra": "forbid"}

-When you use this, only cookies defined by the API can be sent by the client to the API. If other cookie(s) are sent, an error is sent back to the client stating the cookie(s) they sent are not allowed

16. Header Parameter Models

-Used for sending over a group of related header models. Allows you to re-use header model in multiple places.

-Same as any other model class in FastAPI and inherits from *BaseModel*:

```
class CommonHeaders(BaseModel):
    host: str
    save_data: bool
    if_modified_since: str | None = None
```

-Headers, including which are required, show in API docs-Can use *model_config = {"extra": "forbid"}* to forbid extra headers. An error is sent to the client if headers not specified by the API are sent.

-Like with *Cookie*, any *Header* params (including model props) sent with a hyphen in the name are converted to an underscore. To prevent this, use *Header(convert_underscores=False)*

17. Response Model - Return Type

-Python allows you to specify return type via: *def my_func(param_name: type) -> return_type:*

-Any type can be specified as a return type from *int*, to *dict[str, list[float]]*, to *MyModel*

-Should always specify return type for better type checking and function validation. -When specify return type with *FastAPI* endpoint, FastAPI validates return type (raises error if invalid), add return type to JSON schema for response and include in docs, plus serialize returned data to JSON via Pydantic, which makes method faster.

***response_model* Param**
-If you have an endpoint that may need to return multiple types but still want the endpoint to convey a general return type, you can use *response_model* to define the type within *@app* endpoint function wrapper:

```
@app.get("/items/", response_model=list[Item])
async def read_items() -> Any:
    return [
        {"name": "Portal Gun", "price": 42.0},
        {"name": "Plumbus", "price": 32.0},
    ]
```

-This allows FastAPI to use the *response_model* for data documentation, validation, etc., and to convert and filter the output data to its type declaration.-Note, generally better to not do this as using *Any* is often never really preferred, as why even have typing in the first place, then?-If declare return type and *response_model*, FastAPI will prefer *response_model*. This allows FastAPI to still perform data validation and so forth.-If set *response_model=None* to disable creating a response model for the *path* operation

Response Model
-In the same way you can define a model for input params like the request body, you can define a model for the response, then have the endpoint function return it.-Request param and response models can inherit from other user defined models, where the base user model inherits from *BaseModel*.

-Response models are shown in docs**Return a Response Directly**
-Skip this as not well covered here and will be covered later in advanced docs

Redirect Response
-A FastAPI response class that takes in a *URL* param and redirects the user to another page:

```
from fastapi.responses import RedirectResponse
...
@app.get("/teleport")
async def get_teleport() -> RedirectResponse:
    return RedirectResponse(url="https://www.youtube.com/watch?v=dQw4w9WgXcQ")
```

Multiple Return Types
-You may return multiple types, but only if returning either multiple Pydantic types or Python types.-For example, you may not return *Response | dict*

18. Extra Models

-You'll generally have multiple models of the same resource for different layers of the application. Ex. Request model (base model), Response model (returns no password), and repository model (stores hashed password in DB).

.model_dump()
-If you call *.model_dump()* on a pydantic model (ex. *User(BaseModel)*), you get a dict of all the keys/values for props in the model. Ex.: *user_in = UserRecord(username="john", password="secret", email="john.doe@example.com")*
user_dict = user_in.model_dump()   {
'username': 'john',
'password': 'secret',
'email': 'john.doe@example.com',
'full_name': None,
}

-Can then unpack that dict to instantiate a model: *UserRecord(**user_dict)*

-For both actions at once: *UserRecord(**user_in.model_dump())*

-Can pass in extra fields to instantiate a model this way as well:
*UserRecord(**user_in.model_dump(), hashed_password=hashed_password)*

Models from Models
-Given the above, we could do something like

```
# Domain layer
class User(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

# Client passes in plaintext password via request body & https
class UserRequest(User):
    password: str

# Add on hashed password for User as stored in repo
class UserInDB(User):
    hashed_password: str
```

Endpoint Return Variations
-Endpoints can return one of multiple types (ex. *WholeUser | PartialUser*), collections of models, and even responses not made using Pydantic models, like *dict[str, float]*.

19. Response Status Code

-Can set an endpoint to return a specific default status code, if operation succeeds. Ex.

```
@app.post("/items/", status_code=201)
    async def create_item(name: str):
```

-This default status code will then be shown in the docs as the expected status code for a successful response

-HTTP status code review:

- 1xx – “Information.” Rarely used. Cannot have a body.
- 2xx – Success
- 3xx – Redirection
- 4xx – Client error (ex. 404 – resource not found)
- 5xx – Server errors

20. Form Data

-When you need to receive form data instead of JSON fields, you can use the *python-multipart* library (install via *pip*)

-After install, *python-multipart* executes as a runtime dependency behind the scenes, even though you don’t explicitly *import* it anywhere. The library is used to parse file uploads, multi-part bodies, and form submissions.
-FastAPI gets multipart/urlencoded body —> Sends to *python-multipart* —> *python-multipart* parses data into values FastAPI can use with its *Form* or *File* models.

-Not typically used, as REST APIs usually don’t need to read in form data.-Once you install *python-multipart*, you can use *Form* module:

```
from fastapi import Form    ...    @app.post("/login/")
async def login(username: Annotated[str, Form()], password: Annotated[str, Form()]):
    ...
```

-*Form* use example – one way the OAuth2 spec can be used is via the password flow, in which *username* and *password* <u>must</u> be submitted as form fields, and not JSON

-*Form* allows the same configurations set for it as *Query*, *Body*, *Path*, etc. (validations, alias, etc.)

-When you send data to the server from an HTML *<form></form>* entity, the encoding is different from JSON. FastAPI recognizes form submissions as form submissions, and handles them using *Form*.

-When you add an endpoint that takes in *Form*, the docs will show the Request Body type as *application/x-www-form-urlencoded* and give the test data you call the API with through the docs is requested as form data.

21. Form Models

-As with any endpoint param type, you can read in forms using *Pydantic* models, which inherit from *BaseModel*. Define the class and its props. Use it for the *Form* param type in the request. Good practice to do so for request and response models.

```
class FormData(BaseModel):
    username: str
    password: str
```

```
@app.post("/login/")
async def login(data: Annotated[FormData, Form()]):
```

-Force only fields defined by the request input model via model prop: *model_config = {"extra": "forbid"}*

-If you forbid extra form fields from being submitted and a user tries to submit an extra field, an error is raised by FastAPI to the user.

22. Request Files

File
-Can bring in files from client to FastAPI via FastAPI *File*

```
from fastapi import File

@app.post("/files/")
async def create_file(file: Annotated[bytes, File()]):
    return {"file_size": len(file)}
```

-If store file as *bytes* like above, works well for any file (PDF, image, zip, etc.) as all files can be stored as raw sequences of bytes, no extra encoding done. Note, this also stores the whole contents in memory, so only good for small and temporary files.

-When create endpoint that reads in a *File* docs will show button to select and upload file from file explorer

UploadFile
-Can also import *fastapi* module *UploadFile*. This module does not import in *bytes* and only store in memory, but rather imports as a "spooled" file, meaning a file stored in memory up to a max size, but once past that size it is stored on disk.

-Able to get metadata from files stored this way. Can also upload via an *async* interface via python *file-like* library

-Uploading in this manner create a *SpooledTemporaryFile* that can be directly passed to other libraries expecting a file-like object.

-*UploadFile* attributes:

- *filename – str*, original name of file uploaded (ex. *my_file.jpg*)
- *content_type - str*, type of file (*MIME type / media type*) (ex. *image/jpeg*)
- *file – SpooledTemporaryFile*, the actual py file

-*UploadFile async* methods:

- *write(data) – Writes data (str or bytes) to file*
- *read(size) – Reads size(int) bytes/chars in file*
- *seek(offset) – Moves read pointer to offset(int) position in file (ex. my_file.seek(0) for start)*
- *close() – Close file*

-As *UploadFile* methods *async* must *await* when call them

-*UploadFile* will be shown in docs via a button that allows you select file to upload via file browser with data submission type of *multipart/form-data*.

Extra Details
-To make *File* or *UploadFile* optional endpoint request data, declare as | *None = None* second allowed type, as usual

-Can also pass in *Form* and *File* the same validations, *description*, *alias*, etc. fields as other *Param* inheriting models, to give these classes metadata, etc.

Multiple File Uploads
-If want to read in multiple files at once:

```
async def create_files(files: Annotated[list[bytes], File()]):
    ...
async def create_upload_files(files: list[UploadFile]):
...
```

-Or do this wrapping your list of files in *Annotated* to get metadata also:

```
@app.post("/uploadfiles/")
async def create_upload_files(
    files: Annotated[list[UploadFile], File(description="Multiple files as UploadFile")]):
```

23. Request Forms & Files

-Endpoints can take in forms and files at same time:

```
@app.post("/files/")
async def create_file(
    file: Annotated[bytes, File()],
    fileb: Annotated[UploadFile, File()],
    token: Annotated[str, Form()],
):
```

24. Handling Errors

-May reasons why a client error could occur

- Client lack privileges to perform requested operation
- Client lacks access to resource
- Resource doesn’t exist
- etc.

-These typically result in a *4xx* error

-To handle generic client errors, use FastAPI module *HTTPException*

```
from fastapi import FastAPI, HTTPException    @app.get("/items/{item_id}")
async def read_item(item_id: str):
  if item_id not in items:
      raise HTTPException(status_code=404, detail="Item not found")
```

-*HTTP* inherits from Py *Exception* class and provides additional data relevant for APIs

-Note that *raise* it, as with any *Exception*, meaning once exception raised execution halts (unless exception handled)

-When exception raised, user will receive status code in HTTP response and JSON response data showing exception info:

```
{
   "detail": "Item not found"
}
```

***HTTPException* Headers**

-Can also add custom headers to *HTTPException* upon *raise*:

```
raise HTTPException(
    status_code=404,
    detail="Item not found",
    headers={"X-Error": "There goes my error"},
)
```

Where to Raise Exceptions From

-*HTTPException* should always be raised from the endpoint layer as the endpoint layer is the HTTP layer.

-As such, it takes in HTTP requests and the various types of param that can be passed in with a request. it is also for returning a proper response following HTTP standards.-Given this, if an exception occurs during something like searching for an Item in a repo, the error itself needs to be translated into something that makes sense from an HTTP perspective, and thus it is raised from the HTTP communication (endpoint) layer.

Custom Exception Handlers

-For HTTP errors, commonly just raise a handled exception to client via HTTP status code and detail(s)

-For more edge case exceptions, like exception from third-party library, DB/ORM exception, etc., may wish to define your own exceptions and exception handler functions to define and handle such exceptions.

```
class UnicornException(Exception):
   def __init__(self, name: str):
       self.name = name
@app.exception_handler(UnicornException)
async def unicorn_exception_handler(request: Request, exc: UnicornException):
```

```
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something. There goes a rainbow..."},
    )    @app.get("/unicorns/{name}")
async def read_unicorn(name: str):
    if name == "yolo":
        raise UnicornException(name=name)
    return {"unicorn_name": name}
```

-There are three parts to custom exception handling:

- Custom Exception - *UnicornException* – custom exception inheriting from *Exception*
- *@app.exception_handler – Takes in the custom exception. Contains handling logic.*
- *raise – Function where exception is raised. Raises custom exception.*

-The above allows you to thus define your expected error, plus write a function to catch it whenever it is raised. FastAPI will automatically use this function to handle the exception whenever it is raised. The exception can thus be raised from many spots without needing duplicated code.

-Don't confuse *@app.exception_handler* as a route. *@app.get* and *@app.post* are routes but *@app.exception_handler*, while still a wrapper provided by FastAPI, is not an endpoint.

Overriding Default Handlers

-*RequestValidationError* – raised internally by FastAPI when a request contains invalid data. It has a default FastAPI defined exception handler *@app.exception_handler(RequestValidationError)*

-To override *RequestValidationError* handler, we would define a wrapped handling function using the same handler:

```
from fastapi.exceptions import RequestValidationError...
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc: RequestValidationError):
    message = "Validation errors:"
    for error in exc.errors():
        message += f"\nField: {error['loc']}, Error: {error['msg']}"
    return PlainTextResponse(message, status_code=400)
```

-The same can be done for any FastAPI exception, by writing a new handler for *@app.exception_handler(SomeFastAPIException)*

-Could also override to return *JSONResponse*:

```
    return JSONResponse(
        status_code=422,
        content=jsonable_encoder({"detail": exc.errors(), "body": exc.body}),
    )
```

-If do not wish to override existing *@app.exception_handler*, can have your custom exception class inherit from a FastAPI exception, Starlette exception, etc., instead, then only override some fields, only add new fields, etc.

-Reminder: Starlette is the web layer FastAPI is built on top of which contains all HTTP handling. FastAPI wraps starlette's HTTP handling in data, validation, docs, etc. layers on top of this, in part via Pydantic integration, etc.

-*FastAPI*'s *HTTPException* inherits from Starlette's *HTTPException* and NOT FastAPI's *HTTPException*. As such, if override *HTTPException* should override the starlette version of this, so if low level Starlette errors are raised, they can also be caught:

```
from starlette.exceptions import HTTPException as StarletteHTTPException
...
@app.exception_handler(StarletteHTTPException)
async def custom_http_exception_handler(request, exc):        ...
```

25. Path Operation Configuration

Response Status Code

-Can define HTTP code to be returned in success for path operation endpoint wrapper itself:

```
@app.post("/items/", status_code=status.HTTP_201_CREATED)
```

Tags

-Can add tags to path operation endpoint wrapper via:

```
class Item(BaseModel):        tags: set[str] = set()

@app.post("/items/", tags=["items"])
```

-Useful to use with *Enum* to only allowed defined tags:

```
class Tags(Enum):
    items = "items"
    users = "users"

@app.get("/items/", tags=[Tags.items])
async def get_items():
    ...

@app.get("/users/", tags=[Tags.users])
async def read_users():
    ...
```

Path Description

-Can add *doctring* description to path (endpoint) and it will print in docs. A docstring description is often long and takes up multiple lines. It will show in docs at top of endpoint details.

```
@app.post("/items")
async def create_item(item: Item) -> Item:
    """    Long multiple line description
```

"""

-Also can pass in args to *@app* definition of path:
- *summary="Quick summary here."*
- *description="A general length description that gives adequate details"*

Response Description
-Pass in as arg to *@app.action(...)* path definition: *response_description="Description goes here"*

-Response description message shows when success response.

Path Deprecation
-Pass in as arg to *@app.action(...)* path definition: *deprecated=True*

-Endpoint will show in docs but will be grayed out and not able to interact with, to denote as deprecated

26. JSON Compatible Encoder

jsonable_encoder
-A FastAPI provided encoder to convert complex data types, such as Pydantic models, into JSON compatible formats (ex *dict*, *list*)

-Useful for quick serialization of complex objects into DB ready objects

```
from fastapi.encoders import jsonable_encoder    ...
json_compatible_item_data = jsonable_encoder(item)            # returns a dict from an Item
```

json.dumps()
-Review: *import json*

```
json_obj = {"a": "Hello", "b": "World"}
json_str = json.dumps(json_obj)
print(json_str)                                  # {"a": "Hello", "b": "World"}
print(type(json_str))                            # <class 'str'>
```

-JSON module function used to convert a JSON object into a JSON-formatted string.-Can be used to send data over APIs, store structured data as a string, or serialize Python objects into JSON text.

-Can take in a *dict*, *list*, etc. and convert them also into JSON-formatted string

-JSON format is one of the structures NoSQL DB can store

Summary
-Use *jsonable_encoder* to convert pydantic models into JSON compatible formats, like *dict*. Then, if you need a json formatted string, pass the object created by *jsonable_encoder* to *json.dumps* to get a json str.

27. Body - Updates

PUT Updates
-PUT – fully replaces (updates) existing entity

```
@app.put("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:        # replace item fully
```

PATCH Updates
-PATCH – partially replaces (updates) existing entity

-Pydantic models are built to include a *model_dump* function that when called with *exclude_unset=True* that generates a *dict* with only data from props set not by default values w/ default val props excluded:f

```
@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    stored_item_data = items[item_id]                                    # Get repo data
    stored_item_model = Item(**stored_item_data)                         # Put data into model
    update_data = item.model_dump(exclude_unset=True)                    # Get dict
    updated_item = stored_item_model.model_copy(update=update_data)      # Update model
    items[item_id] = jsonable_encoder(updated_item)                      # Update item
    return updated_item
```

-Note use of *model_dump* to get no-default values dict using *model_dump*, then *model_copy* to copy dict of update vals (no default vals) to *updated_existing_user*

-Simplified PATCH steps:
1) Get stored data from repo
2) Put stored data into Pydantic model
3) Get dict of update data endpoint input model
4) Create a copy of stored model, updating fields using input model for update data during copy
5) Convert updated copy model into stored data compatible type
6) Replace existing record
7) Return the updated model

28. Dependencies - Intro

Dependency Injection
-Passing in the functions or classes your functions (or classes) need to work, in contrast to defining that logic inline supports re-usability, interchangeability, and separability. Useful for shared logic, shared DB connections, enforcement of security/auth/role requirements, etc..

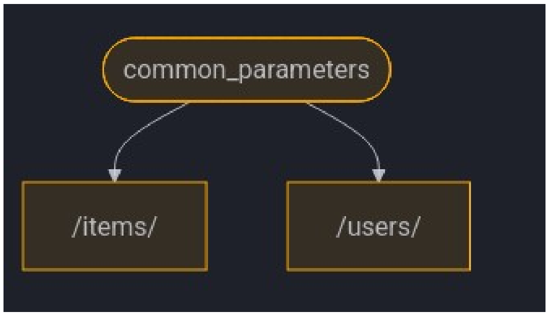
Dependency Injection Functions
-FastAPI makes it simple:
1) Create a dependency (a funct which can take in same params as some path, plus optional extra params)

2) Pass dependency into path funct you want to have access to dependency
3) Access dependency return data, which is output from dependency function on path call

```
from fastapi import Depends, FastAPI
....   async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(commons: Annotated[dict, Depends(common_parameters)]):
    return commons            # returns output of common_parameters

@app.get("/users/")
async def read_users(commons: Annotated[dict, Depends(common_parameters)]):
    return commons            # returns output of common_parameters
```



-When pass dependency from function, FastAPI will take params sent to path function, send them to your *Depends* function, the *Depends* function will then execute its logic on the params, then return them, and then the path function gets the return value to use.

-Additionally, if the dependency takes in a param that the path function does not take in, when you pass that *Depends* into the path, the path now also can take in the params that the *Depends* can take. -Ex. Path function takes in *user_id: int*. Dependency to debug takes in *debug: bool*. Now path function can also take in query param *debug: bool*.

Query vs Depends

*Annotat*ed*[MyQueryModel, Query()]*   # param comes from query params, type MyQueryModel

*Annotat*ed*[FuncReturnType, Depends(func)]* # param comes from func return, type FuncReturnType

-So *Depends* is not that different from *Path* or *Body* or *Query*. When used with *Annotated*, all are just statements that declare where you get your data from and what type that data is, even if the data comes from a back-end user defined function, as in the case of *Depends*.

Depends in Detail

1. Build a shared logic function that takes in the same params as the API paths (extra also allowed)
2. Pass this function into those API endpoints as a *Depends* object
3. The *Depends* function is like an internal API path itself. It reads in like P*ath()*, *Query()*, etc.,

and often has the same FastAPI path signature as the API endpoints which will read it in
4. *Depends* takes in params and performs its internal wrapped function logic on the data, then returns
5. API path functions which took in *Depends* object get the output of that *Depends* function call stored in the *Depends* param itself for typical var use

-Example: Have DB function to query a *User* via *user.id*. If "user not found", raise *HTTPException* 404. This logic happens in multiple endpoints, all which take in only *user.id* as a single path param. Logic gets moved to *Depends* function. Function passed in via *Depends(my_function)* and stored in variable *user*. Now all functions which take in this *Depends* can perform the same user search query w/ potential 404 w/o repeating this code each endpoint.

Depends Var
-Can store a *Depends* function as an *Annotated* reference var where

```
def get_debug(debug: bool = False) -> WishlistDebug | None:
    return WishlistDebug(debug=debug) if debug else None

DEBUGGER = Annotated[WishlistDebug | None, Depends(get_debug)]
```

-Then you can easily import that function variable into wherever you need to inject it

async
-*Depends* functions can be *async* or sync. Is your function one which may need to wait on return? If so, *async*. Otherwise, sync is fine.

-All request declarations, validations and requirements of dependencies and sub-dependencies get integrated into OpenAPI schema docs

Simple Usage
-Path operation functions are declared when a path and operation match. Get this + operation to do so. FastAPI itself takes care of calling the function with the correct params and extracting data from the request.

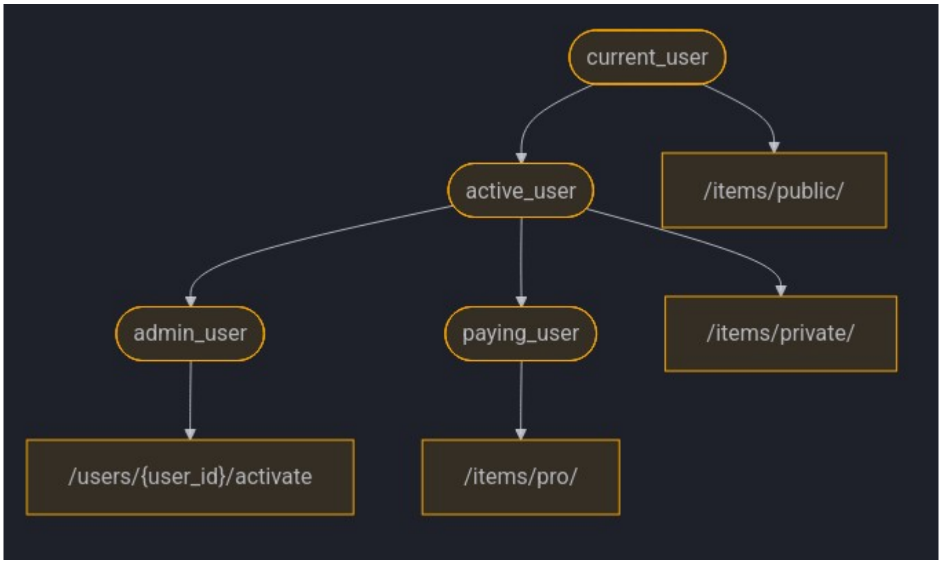
-This is a foundation of web frameworks. The user defines the user interface, the base models, the repo layer to access stored data. What parameters go where for user-defined business logic. The framework does all the underlying essential work, though. Everything from Pydantic models to *Depends* dependency injection. No need to re-build all the middleware and structure. FastAPI already did it. Just define your routes, models, and get moving.

-Dependency injection, resources, providers, services, injectables, components, etc.

FastAPI Compatibility
-FastAPI's dependency injection makes it immediately compatible with all relational DBs, NoSQL DBs, external packages, external APIs, auth and auth sys, API usage monitoring systems, response data injection systems, etc.

-Just swap out the old dependency with the new one

Simple & Powerful




```
def current_user(token: str = Header()) -> User:
    ...  # decode token, look up user
    return user

def active_user(user: Annotated[User, Depends(current_user)]) -> User:
    if not user.is_active:
        raise HTTPException(400, "Inactive user")
    return user

def admin_user(user: Annotated[User, Depends(active_user)]) -> User:
    if not user.is_admin:
        raise HTTPException(403, "Not an admin")
    return user

def paying_user(user: Annotated[User, Depends(active_user)]) -> User:
    if not user.is_paying_customer:
        raise HTTPException(403, "Subscription required")
    return user
```

1. In the above example, current_user is the root dependency function. When FastAPI route /items/pro/ calls paying_user, the dependency functions paying_user depends on are called in order from the root.
2. current_user, as the root, is given token and looks up User
3. *active_user then Depends on current_user and hence gets its User return. It checks user.is_active or raises 400 if inactive*
4. *paying_user then Depends on active_user, and hence gets its User return. It checks user.is_paying_customer and if not raises 403# From this, we see an auth cascade form. The User is passed through a series of checks, where each next check is reached only if the prior one passes and with each new check down the dependency tree, we have the checks of all the former "parent" dependencies passed as well*

29. Dependencies - Classes as Dependencies

-In FastAPI dependencies must be a *Callable*. In Py this means any object you can invoke with *( )*. This includes functions, methods, classes (*MyClass()*), etc..

-To have a class be a *Depends* object, you define the logic you wish executed in the classes *_init_* method. When a FastAPI path is called, the class is then initiated, the logic in *init* runs, and you get access to all *self* var defined in the *init* during object creation.

```
from typing import Callable
...   class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(commons: Annotated[CommonQueryParams,
Depends(CommonQueryParams)]):
    response = {}
    if commons.q:
        response.update({"q": commons.q})
```

-Note that for *Annotated* using *Depends* classes the return type and *Depends* class are both the class type. This allows you access the variables which live in the class, defined in the *init*.

-You can shorthand how you include classes as *Depends*, as well:

```
Annotated[MyClass, Depends(MyClass)]        # long ver
Annotated[MyClass, Depends()]               # short ver
```

-Like with *Depends* functions, classes also get detail OpenAPI schema and docs details

Depends Classes vs Functions

-Prefer functions over classes for dependencies whenever possible

-Use classes for defining *Depends* only when dependency has clear structure, config, re-usable state, and/or grouped params.

-Ex.

```
class RoleChecker:
    def __init__(self, allowed: list[str]):
        self.allowed = allowed
    def __call__(self, user = Depends(get_user)):
        if user.role not in self.allowed:
            raise HTTPException(403)

  admin_only  = RoleChecker(["admin"])          # configured instances
```

```
    staff_only  = RoleChecker(["admin", "staff"])

@app.get("/admin", dependencies=[Depends(admin_only)]): ...
```

30. Dependencies - Sub-dependencies

-Dependencies can be nested across one or more layers

```
def query_extractor(q: str | None = None):
    return q

def query_or_cookie_extractor(
    q: Annotated[str, Depends(query_extractor)],                    # Depends on q_e
    last_query: Annotated[str | None, Cookie()] = None,
):
    if not q:
        return last_query
    return q

@app.get("/items/")
async def read_query(                    # Depends on q_o_c_e
    query_or_default: Annotated[str, Depends(query_or_cookie_extractor)],
):
    return {"q_or_cookie": query_or_default}
```

-See section “28. Dependencies” *current_user* for another good *Depends* on *Depends* chain, also

-Each dependency tree *Depends* is only called once, with its return value stored in cache. To not store value in cache (will be called multiple times), use: *Annotated[str, Depends(get_value, use_cache=False)*

31. Dependencies - Path Operation Decorator Dependencies

-Useful for cases when don’t need return val from dependency *(Depends* inside path operator function) or dependency does not return anything.

-When declare *Depends* in FastAPI path operation decorator, function still executes, but that is all:

```
@app.get("/items/", dependencies=[Depends(verify_token), Depends(verify_key)])
```

-Note *dependencies* is a list of *Depends*. Can take in class or function dependencies.

-Dependencies passed into path operator decorator can do anything a normal dependency does, such as take in *Header()* param, raise *HTTPException*, etc.

-Dependencies passed into path operator decorator can also return something, return val just never discarded

32. Dependencies - Global Dependencies

-Global dependencies will execute for <u>every</u> endpoint in the app whenever it is called.

-Syntax: *app = FastAPI(dependencies=[Depends(verify_token), Depends(verify_key)])*

-Applied as a path operation decorator to all paths in app

33. Dependencies - Yield Dependencies

-When you *yield* from a function instead of *return* your function becomes a generator function. When *yield* from dependency, value is given to function using dependency, just like when *return* from dependency.

-Ex. dependency to get DB: *async def get_db():*

```
    db = DBSession()            # BEFORE  — runs when request starts (setup)
    try:
        yield db                # hands db to endpoint; execution PAUSES here
    finally:
        db.close()              # AFTER   — runs once the response is done (teardown)
```

try
-When use *try* in dependency logic, will receive any exception raised during execution of dependency logic. Can thus handle it and still let *finally* execute.

-Ex. DB object exception occurs at some point during DB use

```
try:
    yield db
except SomeException as e:
    ...
    raise           # raise exception up so FastAPI can handle in exception layer
```

Sub-dependencies
-As typical, dependencies which use *try* and *yield* can depend on other dependencies, as well, including a functions which also *try* and *yield* themselves.

-Example: dependency function which *try* to *yield* an A, where this funct is then used as a dependency by a funct which *try* to *yield* B and uses A to *close* B on *finally*, where this second funct is then also used as a dependency by a third funct which *try* to *yield* C and uses B as a dependency to *close* C on *finally*.

-Functions which *yield* and *return* can depend on each other

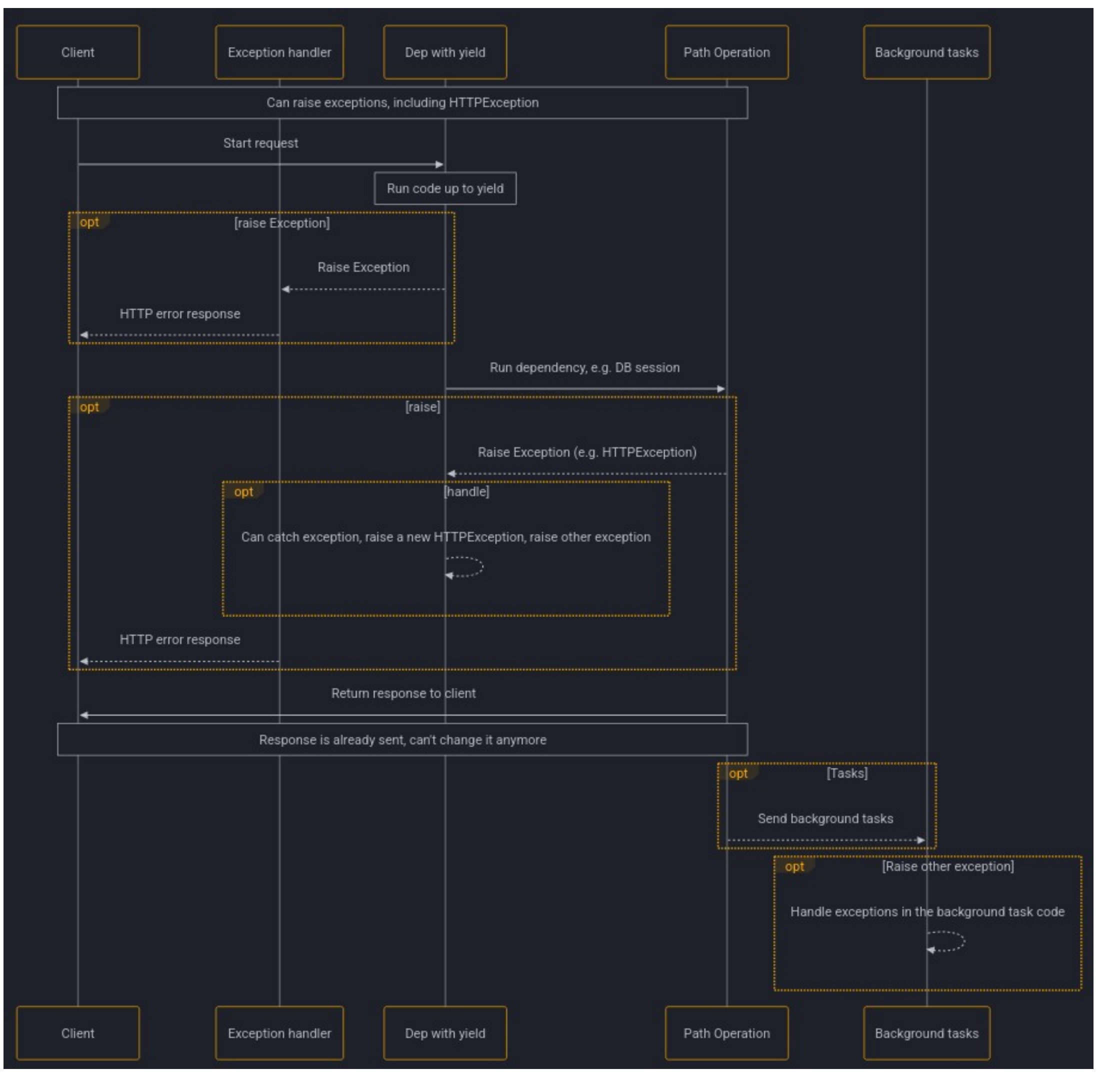
-A single dependency could require multiple dependencies which *yield*

Exceptions
-Useful to catch expected dependency exceptions with *except* also. Make sure to raise them again, though, so they ran be raised in the API layer, also for client, server logs, and record that error even occurred period.

def get_username():
try:
yield "Rick"
except OwnerError as e:
raise HTTPException(status_code=400, detail=f"Owner error: {e}")

-Can raise in python via *raise Exception(...)* but can also simply perform *raise* w/o explicit arg being passed to *raise*. This raises the exact exception caught by *except*. Should either do this or wrap in FastAPI specific exception like *HTTPException.*

yield **Dependency Execution Flow**



Yield Exits
-Typically a dependency that is *yield* closes after response has been sent to client.

-Can choose to close dependency earlier, either as soon as the path operation function ends, but <u>before</u> response is sent to client, as well as explicitly pass in arg to tell set it to not close until after response sent to client.

-Syntax:

```
@app.get("/users/me")    # close before response sent
def get_user_me(username: Annotated[str, Depends(get_username, scope="function")]):

# close after response sent    def get_user_me(username: Annotated[str, Depends(get_username, scope="request")]):
```

-If give a dependency *scope="request"* scope, all sub-dependencies also need *request* scope. If give dep *function* scope sub-dependencies could have *request* or *function* scope. This is because any dependency needs to be able to run its exit code before the sub-dependencies, as that dep might need to still use its sub-dep during its exit code.

Context Managers
-Any Py obj that you can use in a *with* statement. Ex.

```
with open("./somefile.txt") as f:
    contents = f.read()
    print(contents)
```

-Context managers stay open as long as they have something to *yield*, then close automatically. FastAPI dependencies that *yield* are context managers themselves. You can define a custom context manager class with:

```
class MyContextManager:
    def __init__(self):
        self.db = DBSession()

    def __enter__(self):
        return self.db

    def __exit__(self, exc_type, exc_value, traceback):
        self.db.close()

async def get_db():
    with MySuperContextManager() as db:
        yield db
```

34. Security - Intro

-Handling app security without a framework can be a massive undertaking. FastAPI makes it simple.

Auth/Auth Specifications
-OAuth2

- A spec that defines several ways to handle authentication and authorization.
- Extensive. Covers several complex use cases.
- Includes a way to authenticate using a "third party" (login with Facebook, etc.).
- Does not specify how to encrypt comm and expects you to use *HTTPS*.

-OpenID Connect - Another spec which extends OAuth2 and clears up some ambiguities to make it more interoperable. Used in Google login.
-OAuth 1 - An older, more complex spec than OAuth2, since it directly specified how to encrypt comm. Not popular/used today.
-OpenID (not OpenID Connect) - A separate, non-OAuth2-based spec that tried to solve the same problem as OpenID Connect. Not popular/used today.

OpenAPI
-Previously known as "Swagger."

-Open spec for building APIs and now part of the Linux Foundation. FastAPI based on OpenAPI. -Has a way to define multiple security "schemes." Defines the following:

- *apiKey* – app specific key that can come from a query param, header, or cookie

- *http* – standard HTTP authentication systems including *bearer* (a header authorization w/ value of *Bearer* plus a token). Inherited from OAuth2. Also HTTP Basic authentication, HTTP Digest, etc.

- *oauth2* – all the OAuth2 flows to handle security (*implicit*, *clientCredentials*, *authorizationCode*) are for building an OAuth2 provider (like Google, Facebook). *Password grant* is the one flow meant for auth directly in your own app.

- *openIdConnect* – has a way to define how to discover OAuth2 authentication data automatically

FastAPI Utilities
-A handful of *fastapi.security* modules, also, for each of these sec schemes to simplify using them

35. Security – First Steps

FastAPI OAuth2 Password Grant Flow
-FastAPIs most common authentication flow is the OAuth2 Password Grant flow. Allows separation of API from server that does the auth flow (though for FastAPI, it does it itself).

-Flow contains the following steps:

1. Client sends *username* and *password* via <u>form</u> to a token creation/verification endpoint.

2. Server gets user via login form *username*, hashes form *password*, and compares it to the stored hashed user password to verify the user.

3. Server builds a *signature* by feeding a *user data* string (ID + expiration) and the server-side *secret auth key* into a crypto function.

4. The *user data* string and the *signature* are glued together with a period.

5. This is the final *JWT* (JSON Web Token), which is sent back to the client.

6. Client stores *token* locally and sends it back with each new request as *Authorization: Bearer <token>*.

7. Server gets *token* on each request and splits it (at period) into the *user data* string and the *signature*.

8. Server runs that *user data* + *server secret key* through the crypto function again. If the new *signature* matches the client's *signature*, user is authorized.

Password Grant Summary
-Token Build summary:
- user_data_string + server secret auth key → crypto function = signature
- user_data_string.signature = JWT

-Token Verify summary:
- JWT → server → server get user_data from JWT
- server run user_data + server secret auth key → crypto function = signature
- if same signature as JWT, verified

Chapter Summary
-In this FastAPI doc’s chapter we implement a model and URL to read in and verify the format of a token

OAuth2PasswordBearer
-FastAPI module. Defines the internal FastAPI URL that the client passes their token into for verification.

-Covers flow steps:

6. Client stores token locally and sends it back with each new request as *Authorization: Bearer <token>*.

7. Server gets *token* on each request and splits it (at period) into the *user data* string and the *signature*.

8. Server runs that *user data + secret key* through the crypto function again. If the new *signature* matches the client's *signature*, user is authorized.

-Instantiate:

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
 # ex. https://example.com/api/v1/token

- *tokenUrl* does not build a token creation endpoint, it only tells where to send credentials

- *OAuth2PasswordBearer* is callable and thus can be a dependency:
  OAUTH2_SCHEME = Annotated[str, Depends(OAuth2PasswordBearer(tokenUrl="token"))]

  async def read_items(token: Annotated[str, Depends(OAUTH2_SCHEME)]):

-*OAuth2PasswordBearer* looks for *Authorization* header and checks if value is *Bearer <Token>* and returns token as string (stripping away the word *Bearer* from the auth header first.

-If bad header or missing header, responds with *401*.

-Note *OAuth2PasswordBearer* is just for reading in an already created token and verifying it is in valid *Bearer <token>* format. It does not create tokens.

36. Security – Get Current User

Chapter Summary
-In this section we build the signature and token decoding, as well as ability to get and auth user as a dependency.

-Note, this custom signature building, token decoding, etc. will be replaced by *jwt* library later in the *Security* sections.

Signature Building

3. Server builds a *signature* by feeding a *user data* string (ID + expiration) and the server-side *secret auth key* into a crypto function.

```
MOCK_SECRET_KEY = "mock-secret"
TOKEN_EXPIRE = datetime.now() + timedelta(days=2)

def build_signature(user_id: int, secret: str = MOCK_SECRET_KEY,
    expire: datetime = TOKEN_EXPIRE) -> str:
        return f"{user_id}{expire.strftime('%Y%m%d%H%M%S')}{secret}"
```

Token Decoding

8. Server runs that *user data + secret key* through the crypto function again. If the new signature matches the client's signature, user is authorized.

7. Server gets *token* on each request and splits it (at period) into the *user data* string and the *signature*.

```
def decode_token(token: str) -> str | None:
    try:
```

```
        user_id, signature = token.split(".")
    except ValueError:
        return None

    if build_signature(int(user_id)) != signature:
        return None

    return user_id
```

Get User Dependency
-As many endpoints will need to decode token to auth, then get user, and since token is built using *username*, we can create an auth dependency *auth_and_get_current_user* that performs both of these actions and takes in the the *token* as its single arg using the created *OAuth2PasswordBearer* dependency (which reads in a token via *tokenUrl)*:

```
async def auth_and_get_current_user(token: Annotated[str, OAUTH2_SCHEME]) -> User | None:
    username = decode_token(token)
    if username is None:
        return None

    return get_user_by_username(username)
```

-This *auth_and_get_current_user* function can then be passed in as a dependency itself to any function that needs to auth and get user and has a *username* (which token contains) to do so with.

37. Security – Simple OAuth2 w/ Password and Bearer

Chapter Summary
-In this section we build the login endpoint

OAuth2 User / Password
-OAuth2 Password Grant flow requires *user* and *password* to be submitted as form fields. JSON request data will not do, etc. These fields must be named *user* and *password*, too. *user-name* key is invalid.

Scope
-Can also pass in *scope* form field which holds a list of space separated string. Each val in *scope* is generally a security permission.

-Ex. *users:read, users:write, instagram_basic, https://www.google.com/auth/drive*

Get username & password
1. Client sends *username* and *password* via form to a token creation/verification endpoint:

-*OAuth2PasswordRequestForm* is a form with *username*, *password*, *scope*, *grant_type*, *client_id*, and *client_secret* fields, where fields can be referenced as *my_form.prop_name.*

```
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
```

Get User Via Form Data

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    user = get_user_by_username(form_data.username)
    if user is None:
        return HTTPException(status_code=401, detail="Incorrect username or password")
```

Verify Password

-Never store plaintext passwords in DB, only hashed passwords. If store plaintext and DB compromised, uh oh.

-Hashing – pseudo-randomization of a string. Same-in, same-out. Uni-directional (cannot get password from hashed pass).

2. Server gets user via login form username, hashes form password, and compares it to the stored hashed user password to verify the user.:

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    ...

    hashed_form_password = get_password_hash(form_data.password)
    if not verify_password(form_data.password, user_record.hashed_password):
        raise HTTPException(status_code=401, detail="Incorrect username or password")
```

Build Token

4. The user data string and the signature are glued together with a period.

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    ...
    access_token = f"{user_record.id}.{build_signature(user_record.id)}"
```

Return Token

5. This is the final custom token, which is sent back to the client.

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    ...
    return Token(access_token=access_token, token_type="bearer")
```

38. Security – OAuth2 w/ Password (and hashing), Bearer w/ JWT

-The previous “Security” sections completed our password flow custom build. As review, this entailed:

1. Token Build
    - user_data_string + server secret auth key → crypto function = signature
    - user_data_string.signature = JWT

2. Token Verify
    - JWT → server → server get user_data from JWT
    - server run user_data + server secret auth key → crypto function = signature
    - if same signature, verified

as well as an endpoint for getting the user via login form and verifying username and password to serve the token

Chapter Summary
-This section entails turning our custom token into a *jwt* library JWT token and how the previously obfuscated password hashing seen inside the "Verify Password" section of last chapter actually works.

JWT
-JSON Web Token.

-Ex.
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

-JWT is not encrypted, so can be read as plaintext, but is signed so can verify who issued it (ex. your server). This allows you to set an expiring token, then verify it was from you and is valid, until expires.

-Tokens should generally expire eventually, to force user to login again, for proper security.

PyJWT
-Used to generate and verify JWT tokens in Py. Install package *pyjwt* via pip, *requirements.txt*, etc. Reference via *jwt* on *import*.

Password Hashing
-Hashing – pseudo-randomization of a string. Same-in, same-out. Uni-directional (cannot get password from hashed pass)

-Will use package *pwdlib* for password hashing. Recommended algo for this is *argon2*. Install via pip, *requirements.txt*, etc. via: *pwdlib[argon2]*

Hash & Verify Pwd
-Where we define the creation and verification of hashed passwords

```
from pwdlib import PasswordHash

password_hash = PasswordHash.recommended()
DUMMY_HASH = password_hash.hash("dummypassword")

def verify_password(plaintext_pass: str, hashed_pass: str) -> bool:
    return password_hash.verify(plaintext_pass, hashed_pass)
```

```
def get_password_hash(password: str) -> str:
    return password_hash.hash(password)
```

User Auth
2. Server gets user via login form *username*, hashes form *password*, and compares it to the stored hashed user.password to verify the user.

```
def authenticate_user(username: str, password: str) -> bool:
    user_password = get_user_auth_by_username(username)

    if not user_password:
        verify_password(password, DUMMY_HASH)
        return False

    if not verify_password(password, user_password):
        return False

    return True
```

-*authenticate_user* is the main entry point for login auth. This function gets the user's hashed password from the DB then verifies it in one of two ways:

A) If user password is set in DB, verify using form password and stored hashed password. The actual verification is performed by the *password_hash: PasswordHash* instance we created via *PasswordHash.recommended()*.

B) If user password is not set in DB, verify using form password and *DUMMY_HASH*. This is done for security reasons, so response time when when the user doesn't exist is about the same as when it does.

Update Login Endpoint
-Update our endpoint to have it call *authenticate_user*, which calls the PasswordHash *verify* method, instead of touching an auth layer responsibility and breaking layer boundaries:

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    …
    if not verify_password(form_data.password, user_record.hashed_password):
```

BECOMES

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    …
    if not authenticate_user(user_record.username, form_data.password):
```

Build JWT Access Token

5. This is the final *JWT* (JSON Web Token), which is sent back to the client.

-To build a JWT token we first need a random secure secret key. We can generate this from terminal via cmd:

```
$ openssl rand -hex 32
09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7
```

-We then store this in a secret key variable:

```
SECRET_KEY = "7f96f62d5f86f3a43dbe170f9d61e3262a65fe62f0db77d812b4dbfea2acc9be"
```

-We also need an algorithm var:

```
ALGORITHM = "HS512"
```

-And finally a token expiration var:

```
ACCESS_TOKEN_EXPIRATION_MINS = 120
```

-Using this, we can create our token:

```
import jwt

class Token(BaseModel):
    access_token: str
    token_type: str

def create_access_token(data: dict, expires_delta: timedelta | None = None):
    to_encode = data.copy()

    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=90)

    to_encode.update({"exp": expire})

    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

    return encoded_jwt
```

Return Access Token from *login*

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
   …
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(data={"sub": user_record.username})

    return Token(access_token=access_token, token_type="bearer")
```

-*sub* – subject. Standard JWT claim identifying the token owner.

Decode & Verify Token
7. Server gets token on each request and splits it into the user data string and the signature.

8. Server runs that user data + secret key through the crypto function again. If the new signature matches the client's signature, user is authorized.

```
OAUTH2_SCHEME = Annotated[str, Depends(OAuth2PasswordBearer(tokenUrl="token"))]

def decode_token(token: str) -> str | None:
  try:
     payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
     return payload.get("sub")       # username
  except jwt.InvalidTokenError:
     return None
```

-*jwt.decode* – performs token split and signature check

Update Auth Dependency
8. Server runs that user data + secret key through the crypto function again. If the new signature matches the client's signature, user is authorized.

-Also return User. Allows token verification and user get in single call. Can be passed into any routes needing user from token (IE. user from username).

```
async def auth_and_get_current_user(token: OAUTH2_SCHEME) -> User | None:
    username = decode_token(token)
    if username is None:
        return None

    return get_user_by_username(username)
```

Update the /login path operation
-Final */login*:

```
@app.post("/login")
async def login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]) -> Token:
    user_record = get_user_record_by_username(form_data.username)
    if user_record is None:
        raise HTTPException(status_code=401, detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"})

    if not authenticate_user(user_record.username, form_data.password):
        raise HTTPException(status_code=401, detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"})

    access_token = create_access_token(data={"sub": user_record.username})
    return Token(access_token=access_token, token_type="bearer")
```

-Summary: Read in login form → Use form *username* to get *User* → Ensure user exists → authenticate user → create access token → send token to user

-*headers={"WWW-Authenticate": "Bearer"}* – This is the expected auth scheme so client knows how to re-try message. Should be included whenever a *401* means "failed to auth" (bad credentials, token, missing token, etc.)

JWT *sub*
-The subject of the token. Optional to use, but where user identifier goes, so used in example.

-Useful to define a *sub* note just like *"some_username"* but *"username:some_username"* so explicitly know what prop *sub* refers to

FastAPI Auth Docs
-OpenAPI docs give you a nice login for username and password entry.

JWT Scopes
-JWT has additional props, also, like *iss:* for issuer and props that like you define not just authentication (verifying they are who they say they are) but authorization (what they can do) via *scopes*, as well.

-JWT can be passed around. If your bot token has proper auth and *scopes*, no problem. They can call other APIs instead of you!

Security Summary
-The first part of the "Security" sub-sections describes the standard OAuth2 *password grant* authentication flow.

-This auth flow was then implemented by hand, hashing passwords, building signatures, decoding tokens, etc., through custom methods written by me.

-The auth flow is then implemented again, but this time via JWT, using the *pyjwt* (*import jwt*) module. All signature and token creation and verification is handled by the this *pyjwt*, instead.

-The steps in the *password* grant flow are the same both with manual token auth implementation and library assisted JWT token implementation:

1. Client sends username and password via form to a token creation/verification endpoint.

2. Server gets user via login form username, hashes form password, and compares it to the stored hashed user password to verify the user.

3. Server builds a signature by feeding a user data string (ID + expiration) and the server-side secret auth key into a crypto function.

4. The user data string and the signature are glued together with a period.

5. This is the final JWT (JSON Web Token), which is sent back to the client.

6. Client stores token locally and sends it back with each new request as Authorization: Bearer <token>.

7. Server gets token on each request and splits it into the user data string and the signature.

8. Server runs that user data + secret key through the crypto function again. If the new signature matches the client's signature, user is authorized.

39. Middleware

-Middleware – a service (function, class, etc.) that executes with every request before it is processed by any specific path operation, as well as the path response.

-Could do something with request or just run code before request is processed. Same for response.

-Middlware defined by *@app.middleware("http")* FastAPI decorator

```
from app import app

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.perf_counter()
    response = await call_next(request)
    process_time = time.perf_counter() - start_time

    response.headers["X-Process-Time"] = str(process_time)
    return response
```

-Note that just like routes, the *@app* call binds this middleware to your application. This middleware will thus run for *all* routes in the app. Test via docs via *response headers* check.

Middleware Order
-If middleware defined via *@app.middleware,* then executes in reverse order of definition (last defined executes first) on request end, then on response end, first defined executed first

-Can also add middleware via *app.add_middleware(my_middleware)*. Same execution rules apply, except for order in which you called *add_middleware* on.

40. CORS

-Cross-Origin Resource Sharing – Where front-end JS code has code it communicates w/ the back-end did not originate from the back-ends

-Origin – protocol + domain + port. Ex. *http://localhost*, *https://localhost:8080*

Comm Steps
-If front-end running on *http://localhost:8080* and wants to comm w/ *http://localhost* back-end, browser must use an HTTP *OPTIONS* request to back-end with appropriate headers to auth the comm from a diff origin which includes the list of “allowed origins” (ex. *http://localhost:8080)*

Wildcard
-Can declare a wildcard origin list via *

-This sets all origins allowed, but this only applies to certain types of comms, excluding anything realted to credentials (cookies, bearer tokens, etc.)

-Best thus to declare explicit origin list

CORSMiddleware
-Allows you define an allowed origin list, as well as allowed methods, headers, etc.

```
from fastapi.middleware.cors import CORSMiddleware

origins = [
    "http://localhost.tiangolo.com",
    "https://localhost.tiangolo.com",
    "http://localhost",
    "http://localhost:8080",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,                    # list of allowed origins for CORS
    allow_credentials=True,                   # indicates cookies supported for CORS
    allow_methods=["*"],                      # list of HTTP methods allowed for CORS
    allow_headers=["*"],                      # list of headers allowed for CORS
)
```

-Note that if *allow_credentials=True*, then neither *allow methods, headers*, or *origins* can be * and they must be explicitly stated instead

-Also additional args available for *add_middleware* such as *allow_origin_regex, expose_headers, max_age*

41. SQL DB

-FastAPI is built on top of SQLAlchemy. On top of SQLAlchemy is SQLModel, which is built also with Pydantic. SQLModel made by same author as FastAPI.

-SQLModel supports a wide num of SQL DB: PostgreSQL, MySQL, SQLite, Oracle, MS SQL Server, etc.. Docs use SQLite due to single file and as integrated Py support.

-Include in project via library *sqmodel*

SQL Models
-*SQLModel* is defined just lik a a Pydantic *BaseModel* except class inherits from *SQL*Model and *Field*, etc. also imported from *sqlmodel* library:

```
from sqlmodel import Field, Session, SQLModel, create_engine, select
...
class Purchase(SQLModel, table=True):                        # table=true tells is sql table in DB
    id: int | None = Field(default=None, primary_key=True)   # is primary key
    user_id: int = Field(index=True)                         # create SQL index on this col
    item_id: int
    manager_discount: bool | None = False
```

-Note, p-key above can be *int* | *None* to allow us to create *Hero* w/o assigning *id* so DB can assign automatically

SQL Engine
-A *SQLModel* engine defines the connection to a DB

-There is only a single engine object for all instances of your code which connect to the same DB (singleton). Create via:

```
from sqlmodel import create_engine

SQLITE_FILE_NAME = "src/repository/sql/db/database.db"
SQLITE_URL = f"sqlite:///{SQLITE_FILE_NAME}"

CONNECT_ARGS = {"check_same_thread": False}
ENGINE = create_engine(SQLITE_URL, connect_args=CONNECT_ARGS)
```

-*"check_same_thread": False* – required as a single FastAPI request could use more than one thread, thus must allow single request to use more than one threads. Single *SQLModel* session per request handled later.

Create Tables
-Create tables using *create_all* function from *SQLModel*:

```
def create_db_and_tables():
    SQLModel.metadata.create_all(ENGINE)
```

-As simple as that. No writing SQL statements. No SQL terminal. FastAPI does it all for you.

Create Session Dependency
-*Session* – what stores the DB objects in memory and keeps track of any changes needed in the data. Uses DB engine to communicate with DB. Create via:

```
def get_session():
```

```
    with Session(ENGINE) as session:
        yield session

SQL_SESSION = Annotated[get_session, Depends()]
```

-*Session* can then be passed in as dependency to any endpoint that needs DB access

Create Tables On Start-up

```
@app.on_event("startup")
def on_startup():
    create_db_and_tables()
```

-In my tutorial demo work have *main.py*, which also handles all routes and middelware imports, handle DB import, while also still leaving the init of the DB to be owned by the repo layer, I did:

```
# main.py
@app.on_event("startup")
def on_startup():
    init_storage()

# respository/sql/__init__.py
from repository.sql.db.sqlite import create_db_and_tables
def init_storage():
   create_db_and_tables()
```

Create Purchase

```
# repository/sql/models/purchase.py
async def create_purchase(purchase: Purchase, sql_session: SQL_SESSION) -> Purchase:
    sql_session.add(purchase)
    sql_session.commit()
    sql_session.refresh(purchase)          # necessary for purchase object not to be stale
    return purchase

# api/routes/purchase.py
@app.put("/purchases") async def create_purchase(
    purchase: Purchase,
    sql_session: SQL_SESSION) -> Any:

    purchase = SqlPurchase(**purchase.model_dump())
    return await create_purchase(purchase=purchase, sql_session=sql_session)
```

-Note that we put the *create_purchase* method in the repo layer and then pass the *sql_session* dependency to this. The API layer thus uses the same singleton DB instance which the repo layer uses to actually update the repo, as the sql is repo layer owned and the API layer only gets to call methods of it instead of directly touching the session and commiting itself, which should not be an API layer responsibility.

Read Purchases

```
# repo layer
def translate_purchases_to_domain(purchases: Sequence[Purchase]) -> list[DomainPurchase]:
    return [translate_purchase_to_domain(purchase) for purchase in purchases]

# repo layer
async def get_purchases(sql_session: SQL_SESSION, offset: int, limit: int)
 -> list[DomainPurchase]:
    statement = select(Purchase)                          # get sql statement
    statement = statement.offset(offset)                  # get updated statement w/ offset
    statement = statement.limit(limit)                    # get updated statement w/ limit

    result = sql_session.exec(statement)                  # excecute sql statement

    return translate_purchases_to_domain(result.all())    # return all rows in list of domain models

# api layer
@app.get("/purchases")
async def get_purchases(
    sql_session: SQL_SESSION,
    offset: int = 0,
    limit: Annotated[int, Query(le=100)] = 100,
) -> list[Purchase]:

    return await get_sql_purchase(sql_session=sql_session, offset=offset, limit=limit)
```

-Note that we do the getting of the data from the repo in the repo layer, as well as translate from repo model to domain model, then return list of domain model to API, instead of list of SQLModel. This is because following DDD, repo layer can access both domain models and sql models, while API layer should not be directly dealing with SQL models, and should thus be passed domain models, instead.

Read Purchase

```
@app.get("/users/purchases/{purchase_id}")
async def get_purchase(purchase_id: int, sql_session: SQL_SESSION) -> Purchase:
    purchase = await get_sql_purchase(purchase_id=purchase_id, sql_session=sql_session)

    if not purchase:
        raise HTTPException(status_code=404, detail="Purchase not found")

    return purchase
```

Delete Purchase

```
#repo layer
async def delete_purchase(purchase_id: int, sql_session: SQL_SESSION):
        purchase = sql_session.get(Purchase, purchase_id)

        if not purchase:
```

```
        # models/exceptions.py
            raise PurchaseNotFoundException(purchase_id=purchase_id)

        domain_purchase = translate_purchase_to_domain(purchase)

        sql_session.delete(purchase)
        sql_session.commit()

        return domain_purchase

    # api layer
    @app.delete("/purchases")
    async def delete_purchase(purchase_id: int, sql_session: SQL_SESSION) -> Purchase:
        try:
            purchase = await delete_sql_purchase(purchase_id=purchase_id, sql_session=sql_session)
        except PurchaseNotFoundException:
            raise HTTPException(status_code=404, detail="Purchase not found")

        return purchase
```

-Note, once again the repo layer handles the actual deletion of the purchase, commit, etc. All the API layer cares about is making sure repo layer deletes purchase. Does not want to handle deletion itself. Proper DDD again.

Base SQL Models
-In SQLModel, any *SQLModel* class which is defined with *table=True* as seen previously is a table.

-A class which inherits from a *SQLModel* class is also a *SQLModel* class and can define itself as *table=True*, thus making itself a table version of a base class for the business data of the SQL entity, with SQL data for the entity in another model:

```
    class PurchaseBase(SQLModel):
        ‘‘‘ Base class (shared fields) ’’’
        user_id: int
        item_id: int
        manager_discount: bool | None = False

    class Purchase(PurchaseBase, table=True):
        ‘‘‘ DB layer class ’’’
        id: int | None = Field(default=None, primary_key=True, index=True)
        secret_tracking_id: str
```

-For creation of *Purchase*, wish to still allow *secret_tracking_id* to be set, but not *Purchase.id*, as this should be set by DB only:

```
    class PurchaseCreate(PurchaseBase):
        secret_tracking_id: str
```

-For *Purchase* update, make all fields optional and do not allow updating *id*. Note that we inherit from

SQLModel, also, as we should not overwrite already declared props with diff types, as this breaks liskov substitution principle.

```
class PurchaseUpdate(SQLModel):
    user_id: int | None = None
    item_id: int | None = None
    manager_discount: bool | None = False
```

Create Purchase

-For creation of *Purchase*, create Repo layer function to validate *PurchaseCreate* object as a valid *Purchase* via attempting to create a *Purchase* using shared *PurchaseCreate* fields as *Purchase* field input.

-model_validate – Pydantic method. Called on class you desire class in. Returns new instance of that class, build from passed object's fields, with validation run during creation.

-*model_validate* ex. - *Purchase.model_validate(purchase_create)* – reads in a *PurchaseCreate* object and creates *Purchase* object using all fields that match signature across *Purchase* model and *PurchaseCreate* instance.

-*model_validate* called such as during creation of new object, also validates required fields present, types match/coerce, Field constraints hold, custom validators pass, etc. Pydantic, as a package, treats creation + validation as a single step, also. If validation fails, *pydantic.ValidationError* raised.

```
async def create_purchase(
    sql_session: SQL_SESSION, purchase: PurchaseCreate
) -> DomainPurchase:
    db_purchase = Purchase.model_validate(purchase)     # id None as PurchaseCreate has no id

    sql_session.add(db_purchase)
    sql_session.commit()
    sql_session.refresh(db_purchase)            # necessary for purchase object not to be stale

    return translate_purchase_to_domain(db_purchase)
```

-Note also that we return *Domain* purchase, so the repo layer responds with a *model* layer object, which can also be understood by the HTTP (API) layer.

-Return validation happens both on request Purchase (sql *CreatePurchase)* and response Purchase (*/models/Purchase)*

Update Purchase

```
# repo layer
class PurchaseUpdate(SQLModel):
"""Inherits from SQLModel and not PurchaseBase as inheriting from Purchasebase then giving
    same name props would break liskov substitution principle"""

    user_id: int | None = None
```

```
        item_id: int | None = None
        manager_discount: bool | None = False

    # repo layer
    async def update_purchase(
        purchase_id: int, purchase_update: PurchaseUpdate, sql_session: SQL_SESSION
    ) -> DomainPurchase:
        purchase_record = sql_session.get(Purchase, purchase_id)

        if not purchase_record:
            raise PurchaseNotFoundException(purchase_id=purchase_id)

        # do not set values for purchase_update probs with no values set
        purchase_update_data = purchase_update.model_dump(exclude_unset=True)
        purchase_record.sqlmodel_update(purchase_update_data)

        sql_session.add(purchase_record)
        sql_session.commit()
        sql_session.refresh(purchase_record)

        purchase = translate_purchase_to_domain(purchase_record)
        return purchase

# api layer
@app.get("/purchases/user/{user_id}")
async def get_purchases_by_user(user_id: int, sql_session: SQL_SESSION) ->
GetPurchasesResponse:
    purchases = await get_purchases_by_user_id(user_id=user_id, sql_session=sql_session)
    …
    return GetPurchasesResponse(user=user, items_purchased=items_purchased)
```

-As update patches already existing row, no *model_validate* required (no need to build new model)

-Instead use SQLModel method *sqlmodel_update(my_data)* which updates existing data and performs no validation (vlidation already happened when FastAPI parsed request body into PurchaseUpdate).

-*model_dump(exclude_unset=True)* – only include fields the client actually sent. Without, omitted fields come through as defaults and overwrite existing data without unintentionally.

42. Larger Applications

-So far the tutorial (but not my work) has just all been in one global big-ball-of-mud. This section covers basic separation of responsibility and app structure. It defines on such example structure as:

```
├── app                    # package root
│   ├── __init__.py
│   ├── main.py            # module
```

```
│     ├──── dependencies.py
│     └──── routers                # sub-package
│     │  ├──── __init__.py
│     │  ├──── items.py            # sub-module
│     │  └──── users.py
│     └──── internal               # sub-package
│          ├──── __init__.py
│          └──── admin.py          # module
```

APIRouter
-Useful for when an app hosts several domain entities and routes for them. Prevents having to *from app import app* in all endpoint files, which can result in circular dependencies. Lets you define your imports through a FastAPI way, instead of a *main.py* that does all the route imports for you in one file.

-Also useful as allows you to create shared config across routes, as defined by router

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/users/", tags=["users"])
async def read_users():
    ….

@router.get("/users/me", tags=["users"])
async def read_user_me():
    …
```

-All parameters, responses, dependencies, etc. that can be defined on an *@app* route can also be defined on a *@router* route

Shared Router Props
-Can many all routes which implement a single router to all have the shared props:

```
from dependencies.header import SECRET_HEADER
...
router = APIRouter(
    prefix="/items",
    tags=["items", "FastAPI_tutorial"],                # useful for auto interactive docs
    dependencies=[Depends(SECRET_HEADER)],             # effect dependencies only
    responses={404: {"description": "Not found"}}
)
```

-All routes then defined by *@router* will share these props

-Note, *dependencies* in *APIRouter* definition must be *callable*. Any return, yield, etc. result returned from the callable will also be discarded. Dependencies passed in to *APIRouter* should thus be “effect” dependencies only, meaning those which run something, do something, but don’t need to return something.

-d*ependencies* Ex. – a function which adds some header data to *response.headers* but returns nothing

-Ex. All the paths use the same SQL DB? Only define their dependency once via *APIRouter*.

-Imports can be done via standard python shorthand .. or absolute path

-If a *@router* route also defines *tags*, *responses*, etc. itself, those definitions will be <u>added</u> onto the values already defined in *router* declaration, not override them.

```
@router.put(
    "/{item_id}",
    tags=["custom"],                                            # more tags
    responses={403: {"description": "Operation forbidden"}},    # more responses
)
```

Routers in main.py

-While our legacy routing included routes via "it's own ways," via:

```
import api.routes.item
import api.routes.file
…
```

-Now we can include routes in the "FastAPI way":

```
from api.routes.item import router as item_router
...
app.include_router(item_router)
app.include_route(some_other_router)
```

My Personal Architecture

-Unlike the official docs, which largely lacks design even after the current section, I designed my personal tutorial project API using strict Domain-Driven Design principles and layers.

```
├── data                    the data stores themselves, not commited
│   ├── legacy              raw in-memory data (item, purchase, user, wishlist)
│   └── sqlite              database.db, supplied at runtime by the compose volume
│
└── src
    ├── app.py              creates the app's FastAPI instance
    ├── main.py             entry point; wires the app via include_router and route imports
    ├── models              domain objects (Item, User, Purchase, ...), and their exceptions
    │
    ├── api                 the HTTP layer
    │   ├── models          request and response Data Transfer Objects
    │   ├── routes          client-facing endpoints and @app.exception_handler
    │   └── dependencies    reusable Depends shared across routes (headers, debug)
    │
    ├── repository          owns all access to data; no other layer touches a store
```

```
│   ├── __init__.py        init_storage(), the single entry point for storage setup
│   ├── legacy             CRUD access methods for legacy store w/ translations to domain
│   └── sql                CRUD access methods for SQLite store via SQLModel w/ translations to
domain
│       ├── db             engine, session factory, and the SQL_SESSION dependency
│       └── models         table models, translation layer, and the CRUD functions
│
├── security
│   ├── hashing.py         password hashing (isolated from business logic)
│   └── auth               auth, token schemes (jwt, legacy), and role verification
│
└── middleware             cross-cutting request/response work (process time, CORS)
```

-Each layer owns its own models, data access and manipulation, data interpretation, and interface. Each layer aims to be a black box internally, and business objects define the API.

43. Testing

-As previously stated, FastAPI is built in part using Starlette, which is based on HTTPX, which is designed based on Requests. Testing with FastAPI should thus be intuitive.

-FastAPI testing is often done with *pytest*, a testing framework which "makes it easy to write small, readable tests and can scale to support complex functional testing"

-In order to use *TestClient*, as seen below, must first install package *httpx* via *requirements.txt*, *pip*, etc.

***TestClient* Endpoint Test**
-An integration test which exercises the whole stack in-process, everything from routing to response serialization. Not end-to-end, though, as not being tested on an actual deployed system thanks to *TestClient*.

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_get_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello, world"}
```

-Then can run in preferred manner (from IDE w/ python/pytest testing config setup, from cmd line, etc.)

-*TestClient* – allows you to send requests to the API without a server running the app

-Pytest requires all tests to start with *test_* and all test files to also start with *test_*.

Test Separation
-Tests should be properly separated by layer. In DDD, a layer should own it's tests, so it makes sense to have a split like api/tests, repository/tests, etc.

-Standard to test all possible "flows" for an endpoint within reason, including possible "not found", bad param conditions, etc.:

```
def test_get_item_with_query():
    response = client.get("/items/0", params={"q": "apple"})
    assert response.status_code == 200
    assert response.json() == {"item_id": 0, "item_name": "Apple", "query": "apple"}

def test_get_nonexistent_item():
    response = client.get("/items/999")
    assert response.status_code == 404
    assert response.json() == {"detail": "Item not found"}

def test_get_item_query_too_short():
    response = client.get("/items/0", params={"q": "a"})
    assert response.status_code == 422

…
```

***TestClient* data**
-Data passed into to *TestClient* endpoint calles (ex. *Get*) must be in JSON. The response is also best easily read in *JSON*. If you want to test with models, use *pytest* fixtures, which this doc does not cover.

```
def test_create_item():
    response = client.post(
        "/items/",
        json={"id": "foobar", "title": "Foo Bar", "description": "The Foo Barters"},
    )

    assert response.status_code == 200
    assert response.json() == {
        "id": "foobar",
        "title": "Foo Bar",
        "description": "The Foo Barters",
    }
```

44. Debugging

On the Official FastAPI Docs
-The docs teach via running a Uvicorn instance from __*name*__ == *"*__*main*__*"* (on app startup). I built project tutorial with a Dockerfile and launch via compose, though. I thus had Claude Opus 5.0 re-write the docs for me, to teach containerized FastAPI debugging, instead.

Current Setup
-My app is containerized. It runs inside the container, not directly on my local machine.

-If my app was not containerized, the debugger would start an instance of the program itself, then allow debugging through it as I make calls to it, etc..

-As my app is containerized, the app instance running inside the container listens for a debugger to connect to the app, then the debugger can execute through the app.

debugpy
-*debugpy* is a package which is an implementation of the Debug Adapter Protocol for Python. It is written by MS and allows easy container + app + debugger integration into VS Code.

-Setup:

1. Install via package *debugpy* via *requirements.txt*, *pip*, etc.

2. Include into *main.py* & listen for debugger on port 5678 of container

```
import debugpy
import os

if os.getenv("DEBUG") == "1":
    debugpy.listen(("0.0.0.0", 5678))
```

3. Update docker-compose.yml to include added port map & DEBUG to env:

```
services:
uvicorn:
    ports:
     ...
      - "5678:5678"                    # uvicorn port mapping for debugger listening
    environment:
      ...
      - "DEBUG=${DEBUG:-0}"
```

Start Debugging
-Start app from terminal with: *$ DEBUG=1 docker compose up --build -w*

-Per my Dockerfile, I launch my FastAPI *app* from my *main.py* file:
CMD ["uvicorn", "main:app", "--app-dir", "src", "--host", "0.0.0.0", "--port", "8080"]

-On container start-up, Uvicorn imports *main.py*. debugpy.listen() sits at *main.py* module level, so the debugger port opens while *main.py* is still wiring the app, before any request can arrive.

Start Debugger
-To debug code that runs on startup, add *debugpy.wait_for_client()* to *main.py* right after the *listen*

VS Code Setup

1. Open debug panel via *Menu → Run*
2. Click “create a launch.json file”
3. Select “Python Debugger” with “Remote Attach”
4. Set host “localhost” and port “5678”
5. Update path mappings:
```
"pathMappings": [
  {
    "localRoot": "${workspaceFolder}/src",
    "remoteRoot": "/usr/local/app/src"
  }
]
```
6. Select the config in the debug panel and press *F5* to attach debugger

Startup Debugging Manually

-If also want to be able to debug on startup, update *listen* to be:

```
if os.getenv("DEBUG") == "1":
    debugpy.listen(("0.0.0.0", 5678))

    if os.getenv("DEBUG_WAIT") == "1":
        debugpy.wait_for_client()
```

-Then can start via terminal via *$ DEBUG=1 DEBUG_WAIT=1 docker compose up --build -w*

Start Debugging w/ Env Var

-If with to start app with *DEBUG* or *DEBUG_WAIT* automatically, can define these var in *.env* instead of having to pass in manually:

```
DEBUG=1
DEBUG_WAIT=0
```

Step-Debugging & Breakpoints

-Once set up debugger, use in typical manner, by setting breakpoints in code, stepping into, stepping over, inspecting data and flow through call, etc.

45. Response Cookies

***Response* Parameter Cookies**

-Can set cookies via defining endpoint which takes in a response and sets cookie on response.

-When doing this, remember that a FastAPI *Response* param tells the path operator function that "this param is the response," letting you modify the response before the function's return value is serialized into it.

```
# jwt.py
def set_token_cookie(token: str, response: Response) -> Response:
    response.set_cookie(
        key="access_token",
```

```
        value=token,
        httponly=True,
        secure=True,
        samesite="lax",
        max_age=ACCESS_TOKEN_EXPIRATION_MINS * 60,
    )

    return response
```

```
# api/routes/user.py
@app.post("/login")
async def login(
    response: Response,
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
) -> Token:
    ...
    access_token = create_access_token(data={"sub": user_record.username})
    response = set_token_cookie(access_token=access_token, response=response)

    return Token(access_token=access_token, token_type="bearer")
```

-Once set cookie like this, browser stores it for that domain and path and sends it on matching requests. Lifetime depends on max_age/expires; without either, cookie is a session cookie and dies when browser closes.

Return a *Response* Directly
-Can also create a *JSONResponse* from within the route itself then append cookie onto it. Should not be preferred as injection beats composition.

```
@app.post("/cookie/")
def create_cookie():
    content = {"message": "Come to the dark side, we have cookies"}
    response = JSONResponse(content=content)
    response.set_cookie(key="fakesession", value="fake-cookie-session-value")
    return response
```

-If create function like this, MUST return response directly and *content* must be appended onto *Response* explicitly, before return

46. Response Headers

***Response* Parameter Headers**
-In the same way can pass *Response* into endpoint function and then update Cookie header on it before returning response, can also add other headers onto *Response*:

```
@app.get("/headers-and-object/")
def get_headers(response: Response):
```

```
    response.headers["X-My-Header"] = "Some custom header"
    return {"message": "Hello World"}
```

-Headers are tacked onto *Response* just like with cookie so Response includes both response data returned by endpoint method (ex. "message": "Hello World") and headers added onto response.

-Useful also to declare *Response* params in dependencies and set headers (and cookies) in them

-Can also create and return response directly, from within endpoint method:

```
...
    headers = {"X-My-Header": "This is my header text", "Content-Language": "en-US"}
    return JSONResponse(content=content, headers=headers)
```

-Note the use of *"X-"* in naming the above header. Custom headers must be named with a starting "X-".

-If you custom headers that you want client viewable you must add them to your CORS configurations

47. Advanced Dependencies

Parameterized Dependencies

-All dependencies so far have been either functions or classes where FastAPI calls __*init*__

-To make a custom *Callable* class, need to define class with a __*call*__. Whatever is returned from the __*call*__ method is thus what is passed out of your dependency when passed in as a *Depends* param to your path operator decorator function.

Create Debugger Dependency

```
# /api/dependencies/debug.py
class GetDebug:
    def __init__(self):
        self.banned_ip = "23.234.116.75"

    def __call__(self, request: Request, debug = False) -> IPDebugResponse | None:
        if not debug or not request.client:
            return None

        host = request.client.host
        banned_ip = self.banned_ip == host

        return IPDebugResponse(client_ip=host, banned_ip=banned_ip)

IP_DEBUGGER = Annotated[IPDebugResponse | None, Depends(GetDebug())]
```

-*request* is available because __*call*__ declares it as a param; FastAPI injects Request into anything that asks for it

Add Dependency to Route

```python
# /api/routes/wishlist.py
@app.get("/wishlists/user/{user_id}")
async def get_wishlist_by_user_id(
    user_id: int, debugger: IP_DEBUGGER, debug: bool = False
) -> WishlistIPDebugResponse:
    ...
    return WishlistIPDebugResponse(
        wishlist=existing_wishlist, debug=None, ip_debug=debugger
    )
```

-As *IP_DEBUGGER* is passed in a dependency to *get_wishlist_by_user_id*, the dependency declares its own params (*request*, *debug*) and FastAPI fills them from the request

Dependencies w/ *yield*, *HTTPException*, *except*, and BG Tasks
-This was already covered in “33. Dependencies - Yield Dependencies.” See that section’s notes.